BASES DE DADOS PARA APOIO À DECISÃO · 2026/2027

Guia de estudo para o exame

Módulos 1, 2 e 3 — resumo da matéria, exame modelo resolvido e comentado, variações possíveis e um exame de treino

- **Módulo 1** — Modelação Entidade–Relacionamento e modelo relacional
- **Módulo 2** — SQL (CRUD, filtros, junções, agregação, subconsultas)
- **Módulo 3** — Inteligência para Negócios: OLTP/OLAP, Data Warehouse, ETL, modelo dimensional

> **De onde vem este guia.** Foi construído a partir dos **slides das Aulas 1–3**, do **Exame modelo**, da respetiva **Resolução** e do **email da professora**. Tudo o que vai além disso (exemplos extra, variações, exame de treino) foi criado por mim para treino. **Não é uma previsão do exame:** o email avisa que o exame não reproduz o modelo nem garante os mesmos contextos ou distribuição de conteúdos.

> **Como usar.** (1) Lê o resumo de cada módulo. (2) Tapa a solução do exame modelo e resolve-o à mão, a cronometrar. (3) Faz as VARIAÇÃO sem espreitar. (4) Faz o exame de treino (Parte 4) em 90 minutos. (5) Prepara a folha A4 com a Parte 5.

0 · O exame num relance

O que a professora disse e como o exame modelo se encaixa

Regras (email da professora).

- Duração **90 minutos**; cotação **20 valores**.
- **Módulos 1 e 2: 15 valores · Módulo 3: 5 valores.**
- Uma **folha A4 manuscrita** (frente e verso), preparada individualmente: fórmulas, definições, conceitos, apontamentos. **Sem** folhas impressas ou fotocopiadas, nem outros materiais.
- O exame modelo serve para dar uma ideia da estrutura e do tipo de questões — não é uma reprodução do exame.

Mapa do exame modelo

Grupo	Tema	O que pede	Neste guia
I	Modelação ER e modelo relacional M1	Q1 modelo ER (entidades, atributos, chaves, cardinalidades) · Q2 esquema relacional com PK e FK · Q3 chaves candidatas e violação da integridade de entidade	Parte 1
II	SQL M2	Q1 INSERT · Q2 UPDATE com subconsulta · Q3 JOIN + WHERE + ORDER BY · Q4 "sem nenhum…" (anti-junção) · Q5 GROUP BY + HAVING	Parte 2
III	Sistemas analíticos e OLAP M3	Q1 verdadeiro/falso justificado (OLTP, DW, ETL, OLAP) · Q2 modelo dimensional (granularidade, medidas, ≥4 dimensões) · Q3 operações OLAP (drill-down, slice, dice, pivot)	Parte 3

A ligação Grupo ↔ Módulo é **inferida do conteúdo**; o exame modelo não indica a cotação de cada pergunta. O que se sabe pelo email é: M3 = 5 valores, M1+M2 = 15 valores.

Sugestão de gestão do tempo (90 min)

Bloco	Tempo sugerido	Porquê
Grupo I (ER + relacional + chaves)	≈ 25–30 min	O desenho do ER é o que mais tempo come; faz-o limpo à primeira.
Grupo II (SQL)	≈ 30–35 min	5 consultas curtas; mecânicas se tiveres treinado.
Grupo III (BI / OLAP)	≈ 20 min	Vale 5 valores (25 %): não gastes mais tempo do que a cotação justifica.
Revisão	≈ 5 min	Aspas em texto, ponto e vírgula, PK/FK em todas as tabelas, unidades.

É uma sugestão minha, não uma regra: ajusta ao teu ritmo depois de fazeres o exame de treino.

Índice

1 · Módulo 1 — ER e modelo relacional

2 · Módulo 2 — SQL

3 · Módulo 3 — Inteligência para Negócios

4 · Exame de treino (com soluções)

5 · Reta final: folha A4, erros típicos, glossário

Anexos: fontes e limitações · dados de exemplo

1 · Módulo 1

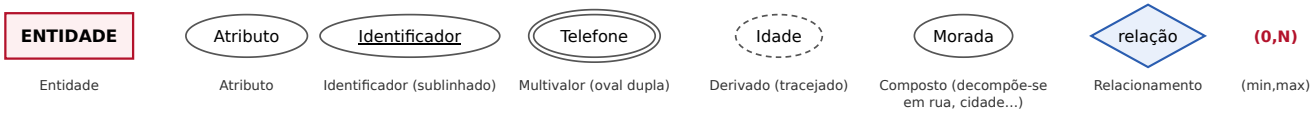
Modelação Entidade–Relacionamento e modelo relacional

1.1 Ideias base

- **Dados** = factos em bruto (42, 1850, 2026). **Informação** = dados com contexto/significado (Idade = 42). Informação útil: *correta, atual, relevante, completa, acessível*.
- **Base de dados** = coleção organizada de dados relacionados, guardada de forma **persistente** (fechar a aplicação ou desligar o computador não apaga os dados).
- **SGBD/DBMS** = software que permite criar, guardar, consultar, atualizar e gerir dados. Funções: armazenamento e acesso, **integridade**, controlo de acesso e segurança, **concorrência**, backup e recuperação. Exemplos: MySQL, PostgreSQL, Oracle, SQL Server, MariaDB, SQLite.
- **Níveis de abstração**: *físico* (como os dados são guardados: ficheiros, índices), *lógico* (que dados e como se relacionam: tabelas, restrições), *vista* (o que cada utilizador/aplicação vê).
- **Do mundo real à BD**: domínio/requisitos → **modelo conceptual (ER)** → **modelo lógico (relacional)** → implementação na BD.
- **Modelar dados** = identificar e representar dados, relações e restrições do domínio: perceber o domínio, tornar relações explícitas, definir regras de negócio, reduzir ambiguidade *antes* de implementar.
- No nível conceptual interessa o **significado e estrutura** dos dados, não tabelas, índices ou armazenamento.

1.2 O modelo Entidade–Relacionamento (ER)

Modelo conceptual com três blocos: **entidades** (coisas/conceitos de interesse: STUDENT), **atributos** (propriedades: Name, DateOfBirth) e **relacionamentos** (associações entre entidades: ENROLLED_IN).



Entidades

Procura no enunciado as "coisas" principais: pessoas, organizações, produtos, serviços, locais, documentos, eventos, transações. **Nem todo o substantivo é uma entidade** — tem de ser algo relevante para o sistema de informação. Pergunta de decisão *atributo ou entidade?* "Este conceito precisa de existir independentemente no modelo?" (partilhado por vários, com propriedades próprias → entidade; senão → atributo).

Tipos de atributos

Tipo	Definição	Exemplo	Notação (aulas)
Atómico / simples	Um único valor por instância	Name, Salary, Birthdate	oval
Multivalor	Um ou mais valores para a mesma instância	Degrees (BSc, MSc, PhD); Telefones	oval dupla · `{Degrees}`
Composto	Hierarquia de atributos	Address = Street + City + Country	decomposto · `Plate(Registration, Country)`
Derivado	Calculado a partir de outros	Age a partir de Birthdate	oval tracejada
Identificador	Distingue cada instância	EID, ISBN	sublinhado

Identificadores e chaves

- **Identificador**: atributo (ou combinação) que identifica **unicamente** cada instância. Toda a entidade deve ter um. Idealmente baseado em informação que **não muda** com o tempo.
- **Chave candidata**: atributo(s) **único(s) e mínimo(s)** (retirar qualquer atributo destrói a unicidade). **Chave primária (PK)**: a candidata escolhida para identificar as instâncias. Ex.: STUDENT(StudentID, Name, Email) — StudentID e Email (se único) são candidatas; escolheu-se StudentID.
- **Identificador composto**: dois ou mais atributos. Típico em **entidades associativas**: ENROLLMENT(StudentID, CourseID, EnrollmentDate) — nenhum sozinho identifica; o par sim, e é mínimo.

1.3 Relacionamentos

Ligação lógica entre entidades. Quatro características a identificar em cada um:

Característica	Significado	Exemplo
Grau	Nº de entidades que participam: **binário** (2), **ternário** (3), **recursivo** (a mesma entidade com papéis diferentes)	CLIENT–order–PRODUCT; STUDENT–enrol–COURSE–SUBJECT; EMPLOYEE *supervises* EMPLOYEE (supervisor/supervisee)
Cardinalidade (máximo)	Quantas instâncias podem participar: **1:1**, **1:N**, **N:M**	PERSON 1:1 PASSPORT · CUSTOMER 1:N ORDER · STUDENT N:M COURSE
Participação (mínimo)	**Total/obrigatória** (mín = 1): toda a instância participa · **Parcial/opcional** (mín = 0)	Todo o aluno está inscrito em ≥1 curso (total); um curso pode não ter alunos (parcial)
Atributos do relacionamento	Factos que descrevem a *associação* e não uma das entidades	StartDate em EMPLOYEE–works on–PROJECT; EstadoDevolucao em EMPRESTIMO–LIVRO

Notação (mín, máx) — a leitura que usamos

O par `(min,max)` escreve-se **junto da entidade** e diz em quantas instâncias do relacionamento **cada instância dessa entidade** participa.

(min,max)	Significado	Exemplo (aulas)
`(0,1)`	opcional, no máximo um	Passport
`(1,1)`	obrigatório, exatamente um	Order → pertence a exatamente um Customer
`(0,N)`	opcional, vários possíveis	Customer → pode fazer zero ou mais Orders
`(1,N)`	obrigatório, vários possíveis	Project → tem pelo menos um Employee

Dica. Lê sempre em voz baixa: "*uma instância de A pode/tem de estar ligada a quantas de B?*". A resposta fala do **outro lado** da relação, mas escreve-se junto de **A**. Ex.: "Um leitor pode ter zero ou mais empréstimos" → `LEITOR (0,N)`; "um empréstimo pertence a exatamente um leitor" → `EMPRESTIMO (1,1)`. Escreve no exame que convenção usas (a resolução oficial usa a mesma: "do lado de EDITORA é 0..N, do lado de LIVRO é 1..1").

1.4 Método para resolver um problema de ER

1. **Sublinha** os substantivos relevantes → candidatas a **entidades**. Descarta os que são só atributos.
2. Para cada entidade, lista os **atributos** do enunciado e escolhe o **identificador** (se o enunciado não dá um, cria um *ID* e diz que o criaste).
3. Classifica atributos "especiais": composto? multivalor? derivado?
4. Sublinha os **verbos** → **relacionamentos**. Para cada um: quem participa? binário/ternário/recursivo?
5. **Cardinalidade máxima** (1:1, 1:N, N:M) a partir de frases como "cada X tem *um único* Y" / "um X pode ter *vários* Y".
6. **Participação mínima** (0 ou 1) a partir de "tem de", "pelo menos um", "pode não ter", "inclui um ou vários".
7. **Atributos do relacionamento**: qualquer dado que só faz sentido para *um par concreto* (ex.: estado de devolução de *um livro num empréstimo*).
8. **Verifica regra a regra**: cada bullet do enunciado deve ter correspondência no teu diagrama.

Atenção. No ER **não há chaves estrangeiras** — as ligações são os losangos. As FK só aparecem no passo seguinte (modelo relacional). Pôr `EditoraID` dentro de LIVRO no ER é um erro frequente.

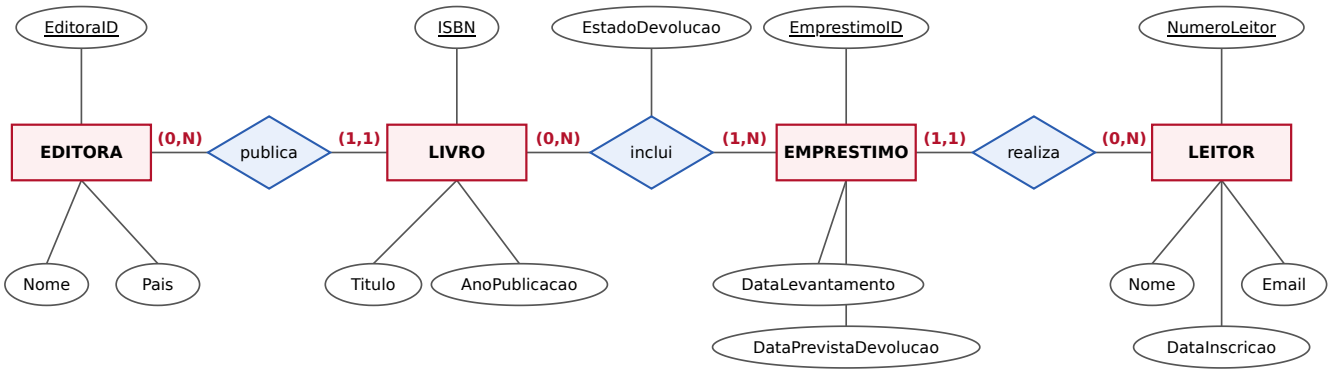
1.5 Exame modelo · Grupo I · Q1 — Modelo ER EXAME MODELO

Enunciado (resumo). Biblioteca municipal: livros, leitores, editoras, empréstimos.

- Livro: ISBN, título, ano de publicação. Cada livro é publicado por **uma única editora**; uma editora pode publicar **vários** livros.
- Editora: identificador, nome, país.
- Leitor: nº de leitor, nome, e-mail, data de inscrição.
- Empréstimo: **um único leitor**, data de levantamento, data prevista de devolução. Pode incluir **vários livros**; um livro pode estar em **vários empréstimos** ao longo do tempo. Para cada livro num empréstimo, regista-se o **estado em que foi devolvido**.

Passo 1 — traduzir cada regra

Regra de negócio	O que modelar
Livro tem ISBN, título, ano	Entidade **LIVRO**(ISBN, Titulo, AnoPublicacao) — o ISBN é o identificador natural
Editora: id, nome, país	Entidade **EDITORA**(EditoraID, Nome, Pais)
Leitor: nº, nome, e-mail, inscrição	Entidade **LEITOR**(NumeroLeitor, Nome, Email, DataInscricao)
Empréstimo: datas	Entidade **EMPRESTIMO**(EmprestimoID, DataLevantamento, DataPrevistaDevolucao). O enunciado não dá identificador → **cria-se EmprestimoID** (a resolução oficial fez o mesmo)
Livro → uma só editora; editora → vários livros	EDITORA–LIVRO **1:N**: EDITORA `(0,N)` · LIVRO `(1,1)`
Empréstimo → um único leitor	LEITOR–EMPRESTIMO **1:N**: LEITOR `(0,N)` · EMPRESTIMO `(1,1)`
Empréstimo inclui vários livros; livro em vários empréstimos	EMPRESTIMO–LIVRO **N:M**: EMPRESTIMO `(1,N)` ("um ou vários") · LIVRO `(0,N)` (pode nunca ter sido emprestado)
Estado em que cada livro foi devolvido	**Atributo do relacionamento** EMPRESTIMO–LIVRO: `EstadoDevolucao` (depende de *um livro específico num empréstimo específico*)



Modelo ER (notação das aulas) para a biblioteca; (min,max) escrito junto de cada entidade.

> **Porque é que EstadoDevolucao não fica em LIVRO nem em EMPRESTIMO?** Em LIVRO teria um único valor por livro (mas o mesmo livro é devolvido em estados diferentes em empréstimos diferentes). Em EMPRESTIMO teria um único valor por empréstimo (mas um empréstimo tem vários livros). Só o **par (empréstimo, livro)** o determina → pertence ao relacionamento N:M.

> **Detalhes aceitáveis.** A resolução diz que "outras respostas equivalentes e corretas devem ser aceites". Por exemplo, EMPRESTIMO (1,N) vs (0,N) para "inclui livros" é discutível — o enunciado diz "um ou vários" (→ 1,N). Se tiveres dúvida, **escreve a tua leitura** do enunciado ao lado.

1.6 Modelo relacional

- **Relação** = tabela: conjunto de **tuplos** (linhas, instâncias) com **atributos** (colunas) definidos; nº de atributos fixo; nome de atributo único; **cada valor é atómico**; a ordem dos tuplos não tem significado. Vocabulário: Tabela = relação · Linha = tuplo · Coluna = atributo.
- **Chave candidata**: conjunto mínimo de atributos que identifica unicamente cada tuplo. **Chave primária (PK)**: a candidata escolhida; valores únicos e **nunca NULL**. **Chave estrangeira (FK)**: atributo(s) que referencia(m) a chave de outra relação — representa a ligação entre tabelas.

Restrições de integridade

Tipo	Regra	Violação típica
Domínio	Cada valor pertence ao domínio do atributo	`Salary = 'abc'` numa coluna numérica
Entidade	A PK identifica unicamente cada tuplo e **não pode ser NULL**	Inserir `ColabID = NULL`; inserir um `ColabID` já existente
Referencial	Um valor de FK ou **coincide** com um valor da chave referenciada, ou é **NULL** (se a relação for opcional)	Inserir livro com `EditoraID = 99` inexistente; apagar uma editora que ainda tem livros

Regras de transformação ER → relacional

No ER	No relacional	Exemplo (aulas)
Entidade	Tabela	—
Atributo	Coluna	—
Identificador	Chave primária	—
1:1	PK de um lado vira **FK no outro** (escolher o lado conforme a participação — habitualmente o lado de participação total)	PERSON(PersonID, Name) · PASSPORT(PassportID, IssueDate, *PersonID*)
1:N	PK do lado "1" vira **FK no lado "N"**	DEPARTMENT(DepartmentID, Name, Location) · EMPLOYEE(EID, Name, Salary, *DepartmentID*)
N:M	**Nova tabela** com as duas FK; a PK é a **combinação** das duas; atributos do relacionamento vão para aqui	EMPLOYEE_PROJECT(EID, ProjectID, StartDate)
Recursivo	FK que referencia a **própria tabela** (pode ser NULL)	EMPLOYEE(EID, Name, Salary, *ManagerID*) com ManagerID → EMPLOYEE(EID)
Atributo **composto**	Cada componente é uma coluna	Address → Street, City, Country
Atributo **multivalor**	Nova tabela com FK para a entidade + o valor	DEGREE(EID, Degree)
Atributo **derivado**	**Não se guarda**; calcula-se	Age calcula-se a partir de Birthdate
Ternário extensão	Tabela própria com as 3 FK (a PK depende das cardinalidades)	ver exercício VI (secção 1.9)

A linha do ternário é uma extensão natural da regra N:M; não está nos slides.

> **Mapa completo da empresa (slide 56).** DEGREE(EID, Degree) · EMPLOYEE(EID, Name, Street, City, Country, Salary, Birthdate, *SupervisorID*, *DepartmentID*) · DEPARTMENT(DepartmentID, Name, Location, *ManagerID*) · PROJECT(ProjectID, Name, Budget, *DepartmentID*) · EMPLOYEE_PROJECT(EID, ProjectID, StartDate).
> Repara: Address → 3 colunas (composto); Age não existe (derivado); Degrees → tabela própria (multivalor); SupervisorID e ManagerID → FK para EMPLOYEE(EID); "gere" 1:1 → FK ManagerID em DEPARTMENT.

1.7 Exame modelo · Grupo I · Q2 — Esquema relacional EXAME MODELO

Aplica as regras tabela a tabela, na ordem: **entidades fortes "do lado 1" primeiro**, depois as do lado N, por fim as tabelas das N:M.

Elemento do ER	Resultado
EDITORA	EDITORA(EditoraID, Nome, Pais)
LIVRO + 1:N com EDITORA	LIVRO(ISBN, Titulo, AnoPublicacao, *EditoraID*) — a PK de EDITORA (lado 1) vira FK em LIVRO (lado N)
LEITOR	LEITOR(NumeroLeitor, Nome, Email, DataInscricao)
EMPRESTIMO + 1:N com LEITOR	EMPRESTIMO(EmprestimoID, DataLevantamento, DataPrevistaDevolucao, *NumeroLeitor*)
N:M EMPRESTIMO–LIVRO	EMPRESTIMO_LIVRO(EmprestimoID, ISBN, EstadoDevolucao) — nova tabela; PK composta; EstadoDevolucao vem do relacionamento

Chaves estrangeiras (a pedir explicitamente).

- LIVRO.EditoraID → EDITORA.EditoraID
- EMPRESTIMO.NumeroLeitor → LEITOR.NumeroLeitor
- EMPRESTIMO_LIVRO.EmprestimoID → EMPRESTIMO.EmprestimoID
- EMPRESTIMO_LIVRO.ISBN → LIVRO.ISBN

Chave primária de EMPRESTIMO_LIVRO = (EmprestimoID, ISBN). Cada um dos dois atributos é *simultaneamente* parte da PK e FK.

Em SQL (caso peçam `CREATE TABLE` — variação provável)

```sql
CREATE TABLE EDITORA (
  EditoraID INTEGER PRIMARY KEY,
  Nome TEXT NOT NULL,
  Pais TEXT
);
CREATE TABLE LIVRO (
  ISBN TEXT PRIMARY KEY,
  Titulo TEXT NOT NULL,
  AnoPublicacao INTEGER,
  EditoraID INTEGER NOT NULL,
  FOREIGN KEY (EditoraID) REFERENCES EDITORA(EditoraID)
);
CREATE TABLE EMPRESTIMO_LIVRO (
  EmprestimoID INTEGER,
  ISBN TEXT,
  EstadoDevolucao TEXT,
  PRIMARY KEY (EmprestimoID, ISBN),
  FOREIGN KEY (EmprestimoID) REFERENCES EMPRESTIMO(EmprestimoID),
  FOREIGN KEY (ISBN) REFERENCES LIVRO(ISBN)
);
```

`NOT NULL` em `LIVRO.EditoraID` traduz o `(1,1)` (participação obrigatória). Se fosse `(0,1)`, a FK poderia ser NULL.

Erros frequentes. (1) Esquecer a tabela da N:M ou pôr uma FK "de livros" dentro de EMPRESTIMO. (2) Esquecer **EstadoDevolucao**. (3) Pôr a FK no lado errado do 1:N (a FK vai sempre para o lado N). (4) PK simples em EMPRESTIMO_LIVRO (um empréstimo teria só um livro). (5) Não assinalar as FK.

1.8 Exame modelo · Grupo I · Q3 — Chaves e integridade de entidade

EXAME MODELO

> COLABORADOR(ColabID, Nome, Email, NIF, Departamento). **Q3:** indique duas possíveis chaves candidatas e dê um exemplo de violação da integridade de entidade.

- **Chaves candidatas:** `ColabID` e `NIF` — identificam univocamente cada colaborador, são mínimas (1 atributo) e assume-se que não admitem NULL. (`Email` também serve *se* for único.)
- **Não são** candidatas: `Nome` (há homónimos) e `Departamento` (repete-se). Um conjunto como `{ColabID, Nome}` é único mas **não é mínimo** (super-chave).
- **Porque é que só uma é "a" PK?** A PK é a candidata *escolhida* (aqui ColabID); as restantes ficam como chaves alternativas (UNIQUE).
- **Violação da integridade de entidade:** inserir um colaborador com `ColabID = NULL` (a PK não pode ser nula) **ou** inserir um `ColabID` já existente (PK duplicada).

> **Dica.** Resposta completa em 3 linhas: definição de candidata (única + mínima) → duas candidatas com justificação → exemplo concreto `INSERT … VALUES (NULL, 'Ana', …)`. Dar a definição não custa nada e mostra domínio.

1.9 Variações possíveis e exercícios

> **O que a professora pode mudar sem sair da matéria.**
> - Outro **domínio** (hospital, escola, loja, transportes...), com outras regras de negócio.
> - Outros **tipos de relacionamento**: 1:1 (com participação opcional), **recursivo**, **ternário**.
> - Outros **tipos de atributo**: composto (morada), multivalor (telefones), derivado (idade).
> - Pedir **o contrário**: dado um esquema/diagrama, identificar chaves, FK, cardinalidades ou violações de integridade.
> - Pedir **CREATE TABLE** em vez de esquema abstrato; pedir **justificação** ("porquê tabela própria?").
> - Pedir **integridade referencial** (em vez de entidade) ou **domínio** na pergunta de chaves.

Caso A — Clínica veterinária (composto + multivalor + derivado + 1:N + N:M com atributos)

> **Enunciado.** Uma clínica guarda os **donos** (NIF, nome, morada = rua, cidade e código postal, e vários telefones) e os **animais** (nº de chip, nome, espécie, data de nascimento; a idade é consultada com frequência). Cada animal pertence a um único dono; um dono pode (ainda) não ter animais. Os **veterinários** têm cédula profissional e nome. Uma **consulta** é feita por um veterinário a um animal numa data/hora e tem um diagnóstico; um animal pode ter várias consultas e ser visto por vários veterinários. Faça o ER e o esquema relacional.

ER

- DONO(NIF, Nome, **Morada**(Rua, Cidade, CodPostal), **{Telefone}**)
- ANIMAL(ChipID, Nome, Especie, DataNascimento, *Idade* derivado)
- VETERINARIO(Cedula, Nome)
- DONO–possui–ANIMAL: **1:N**, DONO (0,N), ANIMAL (1,1)
- ANIMAL–consulta–VETERINARIO: **N:M**, ambos (0,N); atributos **DataHora, Diagnostico**

RELACIONAL

- DONO(NIF, Nome, Rua, Cidade, CodPostal)
- DONO_TELEFONE(NIF, Telefone) — FK NIF→DONO
- ANIMAL(ChipID, Nome, Especie, DataNascimento, *NIF*) — FK NIF→DONO
- VETERINARIO(Cedula, Nome)
- CONSULTA(ChipID, Cedula, DataHora, Diagnostico) — FK ChipID→ANIMAL, Cedula→VETERINARIO

> **Porquê?** Morada composta → 3 colunas. Telefones multivalor → tabela própria (cada valor atómico). Idade derivada → **não** se guarda. **DataHora entra na PK de CONSULTA** porque o mesmo animal pode ver o mesmo veterinário várias vezes; sem ela, só haveria uma consulta por par.

Caso B — Universidade (N:M com atributo, 1:1, recursivo)

> **Enunciado.** **Alunos** (nº mecanográfico, nome) inscrevem-se em **unidades curriculares** (código, nome, ECTS); cada inscrição tem **ano letivo** e **nota** (pode ainda não existir). Cada UC é **lecionada** por um docente (ID, nome); um docente leciona várias UC. Cada UC tem **exatamente um coordenador** (docente) e um docente coordena, no máximo, uma UC. Cada docente pode ter um **docente orientador**.

ER

- ALUNO(NumAluno, Nome) · UC(CodUC, Nome, ECTS) · DOCENTE(DocenteID, Nome)
- ALUNO–inscreve-se–UC: **N:M**, atributos AnoLetivo, Nota; ALUNO (0,N), UC (0,N)
- DOCENTE–leciona–UC: **1:N**; DOCENTE (0,N), UC (1,1)
- DOCENTE–coordena–UC: **1:1**; DOCENTE (0,1), UC (1,1)
- DOCENTE–orienta–DOCENTE: **recursivo** 1:N; orientador (0,N), orientado (0,1)

RELACIONAL

- ALUNO(NumAluno, Nome)
- DOCENTE(DocenteID, Nome, *OrientadorID*) — FK→DOCENTE, pode ser NULL
- UC(CodUC, Nome, ECTS, *DocLecionaID*, *DocCoordID*) — duas FK→DOCENTE; **DocCoordID UNIQUE**
- INSCRICAO(NumAluno, CodUC, AnoLetivo, Nota)

> **Porquê?** No 1:1 a FK vai para o lado de **participação total** (UC tem sempre coordenador) e fica `UNIQUE`, garantindo que um docente não coordena duas UC. **AnoLetivo na PK** permite repetir a UC noutro ano; se o enunciado dissesse "só uma inscrição por aluno e UC", a PK seria (NumAluno, CodUC). **Nota** pode ser NULL.

Caso C — "Que violação é esta?" (integridade)

Dadas DEPARTAMENTO(DepID, Nome) com {(1,TI), (2,RH)} e FUNCIONARIO(FuncID, Nome, NIF, *DepID*) com {(10,Ana,111,1), (11,Rui,222,2)}, onde DepID → DEPARTAMENTO. Diz se cada operação viola alguma restrição:

Operação	Resposta
a) `INSERT INTO FUNCIONARIO VALUES (10,'Eva','333',1)`	**Entidade**: FuncID = 10 já existe (PK duplicada).
b) `INSERT INTO FUNCIONARIO VALUES (12,'Eva','333',3)`	**Referencial**: não existe departamento 3.
c) `INSERT INTO FUNCIONARIO VALUES (NULL,'Eva','333',1)`	**Entidade**: PK nula.
d) `INSERT INTO FUNCIONARIO VALUES (13,'Eva','333',NULL)`	**Nenhuma** — uma FK pode ser NULL se a ligação for opcional (se DepID fosse NOT NULL, violava a regra de obrigatoriedade).
e) `DELETE FROM DEPARTAMENTO WHERE DepID = 1`	**Referencial**: a Ana referencia o departamento 1. O SGBD rejeita (ou exige uma ação definida: CASCADE, SET NULL).
f) `INSERT INTO FUNCIONARIO VALUES ('x','Eva','333',1)` com FuncID INTEGER	**Domínio**: valor de tipo errado.

Testado em SQLite com `PRAGMA foreign_keys=ON`: a inserção de um livro com `EditoraID = 99` e o `DELETE` de uma editora com livros falham com *FOREIGN KEY constraint failed*; um `ColabID` duplicado falha com *UNIQUE constraint failed*.

Exercícios das aulas (Exercícios I–VI) — soluções sugeridas

Os enunciados dos slides estão em inglês (e algumas passagens ficam ambíguas). As soluções abaixo são **uma possibilidade razoável** com as suposições indicadas — o importante é o raciocínio.

I · ArtBase (galeria de arte)

ER	Esquema relacional
ARTISTA(Nome, LocalNascimento, Idade, Estilo) · OBRA(Titulo, Ano, Tipo, Preco) · GRUPO(NomeGrupo) · CLIENTE(Nome, Morada, TotalGasto). ARTISTA–cria–OBRA **1:N**: artista (0,N), obra (1,1). OBRA–pertence–GRUPO **N:M** (uma obra pode estar em vários grupos). CLIENTE–gosta–ARTISTA **N:M**. CLIENTE–gosta–GRUPO **N:M**.	ARTISTA(Nome, LocalNascimento, Idade, Estilo) · OBRA(Titulo, Ano, Tipo, Preco, *NomeArtista*) · GRUPO(NomeGrupo) · OBRA_GRUPO(Titulo, NomeGrupo) · CLIENTE(NomeCliente, Morada, TotalGasto) · CLIENTE_ARTISTA(NomeCliente, NomeArtista) · CLIENTE_GRUPO(NomeCliente, NomeGrupo).

Notas: "nomes únicos" → Nome serve de PK. Idade é derivável da data de nascimento, mas o enunciado dá a idade. `GROUP` é palavra reservada em SQL — usa outro nome na tabela.

II · Livraria (clientes, compras, editoras)

ER	Esquema relacional
CLIENTE(CodCliente, Morada, Telefone, Tipo[singular/coletiva]) · LIVRO(ISBN, Autor, Assunto, Stock) · EDITORA(CodEditora, Morada, Telefone, NomeGestor). CLIENTE–compra–LIVRO **N:M** com atributo **Data**: cliente (0,N), livro (0,N). EDITORA–fornece–LIVRO **1:N** ("editoras diferentes não fornecem o mesmo livro"): editora (0,N), livro (1,1).	CLIENTE(CodCliente, Morada, Telefone, Tipo) · EDITORA(CodEditora, Morada, Telefone, NomeGestor) · LIVRO(ISBN, Autor, Assunto, Stock, *CodEditora*) · COMPRA(CodCliente, ISBN, Data).

Se o mesmo cliente puder comprar o mesmo livro em dias diferentes, a **Data entra na PK**; se só puder uma vez, a PK é (CodCliente, ISBN).

III · Hospital universitário — enunciado confuso; serve para treinar a identificação de entidades. Esboço: HOSPITAL(CNPJ) tem várias ALA (ala pertence a **um** hospital → 1:N); ENFERMEIRO(CRE, Nome) pertence a **uma** ala (N:1) e reporta a um **enfermeiro-chefe** (**recursivo**); MEDICO(CRM, Nome, Especialidade); PACIENTE(CodUtente, Nome) com PLANO_SAUDE(CodPlano, Nome, Telefone) (N:1); ATENDIMENTO = N:M MEDICO–PACIENTE com atributo **DataHora**; EXAME/ANALISE ligado a PACIENTE. "Todos os empregados têm contrato exclusivo" sugere uma generalização (EMPREGADO → médico/enfermeiro) que **não é matéria dos slides**.

IV · Distribuição por camiões

ER	Esquema relacional
ARMAZEM(NumArmazem, **Localizacao**(Cidade, Rua, Numero)) · LOJA (mesma informação) · TIPO_CAMIAO(CodTipo, CapacidadeVolume, CapacidadePeso) · CAMIAO(Matricula, NumCamiao, Quilometros) · VIAGEM(NumViagem, DataPartida) · ETAPA(NumEtapa dentro da viagem) · PACOTE(NumPacote, Volume, Peso, PrazoEntrega). CAMIAO–é_de–TIPO **N:1**; VIAGEM–usa–CAMIAO **N:1**; VIAGEM–parte_de–ARMAZEM **N:1**; ETAPA–pertence–VIAGEM **N:1** e ETAPA–termina_em–LOJA **N:1**; PACOTE–entregue_em–ETAPA **N:1** (suposição).	ARMAZEM(NumArmazem, Cidade, Rua, Numero) · LOJA(NumLoja, Cidade, Rua, Numero) · TIPO_CAMIAO(CodTipo, CapVolume, CapPeso) · CAMIAO(Matricula, NumCamiao, Quilometros, *CodTipo*) · VIAGEM(NumViagem, DataPartida, *NumArmazem*, *Matricula*) · ETAPA(NumViagem, NumEtapa, *NumLoja*) · PACOTE(NumPacote, Volume, Peso, Prazo, *NumViagem, NumEtapa*).

Ponto-chave: o nº da etapa só identifica dentro da viagem → **PK composta (NumViagem, NumEtapa)**, e a FK de PACOTE para ETAPA é **composta**.

V · Lar de idosos (torneios)

ER	Esquema relacional
PARTICIPANTE(CodPart, Nome, **DataNasc**(Dia, Ano), *NumJogos* derivado) · JOGO(NomeJogo, Tipo[pares/individual], DataSaida, *NumParticipantes* derivado) · PARTIDA(CodPartida, PontosEquipa). PARTICIPANTE–participa–JOGO **N:M**: participante (0,N), jogo (**2**,N) ("não há jogos de um só jogador"). JOGO–tem–PARTIDA **1:N**: jogo (1,N), partida (1,1).	PARTICIPANTE(CodPart, Nome, DiaNasc, AnoNasc) · JOGO(NomeJogo, Tipo, DataSaida) · PARTICIPA(CodPart, NomeJogo) · PARTIDA(NomeJogo, CodPartida, PontosEquipa).

Armadilha: "nº de jogos em que participou" e "nº total de participantes" são **derivados** (contagens) — não se guardam. A partida só existe dentro de um jogo → PK composta. Se os pontos fossem por equipa, precisarias de uma tabela de pontos (multivalor).

VI · Pessoas, grupos, anos e cidades (ternário)

ER	Esquema relacional
PESSOA(Nome, Idade) · GRUPO(Nome) · ANO(Ano) · CIDADE(Nome). **Ternário** PESSOA–forma–GRUPO–ANO: em cada ano, cada pessoa pertence a **exatamente um** grupo (o enunciado diz que colocar o Ano como entidade limita os grupos por ano). **Ternário** GRUPO–viaja–ANO–CIDADE: em cada ano, cada grupo visita **pelo menos uma** cidade.	PESSOA(NomePessoa, Idade) · GRUPO(NomeGrupo) · ANO(Ano) · CIDADE(NomeCidade) · FORMA(NomePessoa, Ano, *NomeGrupo*) · VIAJA(NomeGrupo, Ano, NomeCidade).

Ponto-chave: em FORMA a PK é (Pessoa, Ano) — não inclui o Grupo, porque "um grupo por pessoa e ano" é o que a restrição 1 impõe. Em VIAJA os três atributos juntos formam a PK (vários pares grupo-ano-cidade).

2 · Módulo 2

SQL: criar, consultar, atualizar, apagar, juntar, agregar

> **Base de dados usada nos exemplos.** Para os exemplos de SQL uso o esquema do **Grupo II** do exame modelo (manutenção de bicicletas), preenchido com dados fictícios (anexo B). Todos os resultados mostrados foram obtidos **a executar as consultas em SQLite** — por isso são fiáveis, mas os *dados* são inventados.
> TECNICO(TecnicoID, Nome, Certificacao, AnosExperiencia, Especialidade) · MANUTENCAO(ManutencaoID, Descricao, *TecnicoID*, Ano, Tipo, Custo) · CATEGORIA(CategoriaID, NomeCategoria) · MANUTENCAO_CATEGORIA(ManutencaoID, CategoriaID)

2.1 CRUD e criação de tabelas

CRUD = as quatro operações fundamentais sobre dados persistentes: **C**reate (`INSERT`), **R**ead (`SELECT`), **U**pdate (`UPDATE`), **D**elete (`DELETE`).

```sql
CREATE TABLE employees (
  id INTEGER PRIMARY KEY AUTOINCREMENT,   -- chave primária gerada automaticamente
  name TEXT NOT NULL,                     -- obrigatório
  department TEXT,
  salary REAL
);
CREATE TABLE employees2 (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  dept_id INTEGER,
  FOREIGN KEY (dept_id) REFERENCES departments(id)   -- chave estrangeira
);
```

Tipos usados nas aulas (SQLite): `INTEGER`, `TEXT`, `REAL`. PK composta: `PRIMARY KEY (ColA, ColB)` no fim da definição (ver 1.7).

INSERT · UPDATE · DELETE

```sql
INSERT INTO employees (name, department, salary)
VALUES ('Jane Doe', 'Engineering', 75000.00);

UPDATE employees
SET salary = 65000.00
WHERE name = 'John Smith';

DELETE FROM employees
WHERE name = 'Jane Doe';
```

> **Atenção.** `UPDATE` e `DELETE` **sem `WHERE` afetam todas as linhas**. É a observação mais repetida nos slides: “always use a WHERE clause!”. Textos entre **aspas simples**; números sem aspas; lista de colunas e lista de valores na **mesma ordem**.

2.2 SELECT: a ordem das cláusulas

```
SELECT   [DISTINCT] colunas / expressões / agregações
FROM     tabela [AS alias]
[JOIN    outra_tabela [AS alias] ON condição_de_junção]
[WHERE   condição sobre linhas]
[GROUP BY colunas]
[HAVING  condição sobre grupos]
[ORDER BY colunas [ASC|DESC]]
[LIMIT   n];
```

Ordem em que o SGBD "pensa" (útil para perceber erros): **FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT**. Por isso o `WHERE` não pode usar funções de agregação (os grupos ainda não existem) e o `HAVING` sim.

Operadores de filtro

Operador	Uso	Exemplo
`= <> > >= < <=`	comparações	`Custo > 100`
`AND · OR · NOT`	combinar condições (usa parênteses com OR)	`Tipo='Corretiva' AND Ano=2026`
`BETWEEN a AND b`	intervalo **inclusivo**	`amount BETWEEN 500 AND 1000`
`IN (…)`	um de vários valores (alternativa limpa a vários OR)	`region IN ('North','East')`
`LIKE`	**%** = zero ou mais caracteres · **_** = exatamente um	`name LIKE 'A%'`
`IS NULL / IS NOT NULL`	testar valores em falta (**nunca** `= NULL`) extra	`Certificacao IS NULL`
`DISTINCT`	remove linhas repetidas do resultado	`SELECT DISTINCT region …`
`ORDER BY … DESC`	ordenar (ASC por defeito)	`ORDER BY amount DESC`
`LIMIT n`	restringe o nº de linhas (SQLite). Em SQL Server é `TOP n`	`… LIMIT 3`

extra = não está explicitamente nos slides mas é indispensável e aparece nas soluções ou nas variações naturais.

2.3 JOIN: combinar tabelas

- `JOIN … ON` (= INNER JOIN) liga linhas de duas tabelas quando a condição se verifica — normalmente **FK = PK**. Linhas sem correspondência **desaparecem**.
- `LEFT JOIN` mantém **todas as linhas da tabela da esquerda**; onde não há correspondência, as colunas da direita vêm a NULL. Usa-se para "mesmo os que não têm…" e para anti-junções.
- **Aliases** (`FROM MANUTENCAO AS M`) encurtam e resolvem ambiguidades: se duas tabelas têm a coluna `TecnicoID`, escreve `M.TecnicoID`.
- Relações **N:M** precisam de passar pela tabela de associação (ex.: MANUTENCAO → MANUTENCAO_CATEGORIA → CATEGORIA): dois JOIN seguidos.

> **Como o slide decompõe uma consulta com JOIN + WHERE.** `FROM employees`: começa na tabela principal → `JOIN departments ON …`: junta cada linha ao departamento correspondente → `WHERE …`: filtra a tabela "larga" resultante → `SELECT …`: escolhe as colunas finais.

2.4 Agregação: GROUP BY e HAVING

- Funções: `COUNT`, `SUM`, `AVG` (nos slides), mais `MIN` / `MAX` extra.
- `COUNT(*)` conta **linhas**; `COUNT(col)` conta valores **não NULL**; `AVG` / `SUM` ignoram NULL.
- `GROUP BY` junta linhas com o mesmo valor em "linhas-resumo". **Regra:** toda a coluna no `SELECT` que *não* seja agregada tem de estar no `GROUP BY`.

	WHERE	HAVING
Filtra	**linhas** individuais, *antes* de agrupar	**grupos**, *depois* de agrupar
Agregações	não pode usar (*misuse of aggregate*)	pode e costuma usar
Exemplo	`WHERE Ano = 2026`	`HAVING COUNT(*) >= 2`

2.5 Subconsultas

Forma	Quando usar
IN (SELECT ...)	"pertence ao conjunto de..." — usada no UPDATE do exame modelo.
EXISTS / NOT EXISTS	"existe / não existe nenhuma linha relacionada...". **Correlacionada**: a subconsulta refere a tabela de fora. `SELECT 1` chega — só interessa se há linhas.
Subconsulta escalar	devolve **um** valor: `Custo > (SELECT AVG(Custo) FROM …)`.
NOT IN	Cuidado: se a subconsulta devolver **algum NULL**, `NOT IN` não devolve **nenhuma** linha (ver armadilha abaixo).

2.6 Exame modelo · Grupo II resolvido EXAME MODELO

Q1. Inserir um técnico: T15, Rita Lopes, certificação Shimano, 6 anos, especialidade Travões.

Como pensar. Tabela TECNICO, 5 colunas → `INSERT INTO … (colunas) VALUES (…)`. Texto entre aspas simples (inclusive `'T15'`, que é texto), 6 sem aspas, mesma ordem nas duas listas.

```
INSERT INTO TECNICO
  (TecnicoID, Nome, Certificacao, AnosExperiencia, Especialidade)
VALUES
  ('T15', 'Rita Lopes', 'Shimano', 6, 'Travões');
```

Estado da tabela depois de executar:

TecnicoID	Nome	Certificacao	AnosExperiencia	Especialidade
T15	Rita Lopes	Shimano	6	Travões

Erros típicos. `T15` sem aspas · 6 entre aspas · valores por ordem diferente das colunas · esquecer o acento em *Travões*.

Q2. Aumentar 10 % o custo das intervenções do tipo *Corretiva* realizadas por técnicos cuja especialidade é *Transmissão*.

Como pensar. O que se altera (Custo) e o tipo estão em **MANUTENCAO**; a especialidade está em **TECNICO**. Em SQL padrão o `UPDATE` altera **uma só tabela**, por isso o filtro pela especialidade faz-se com uma **subconsulta**: "o TecnicoID está no conjunto dos técnicos de Transmissão". Aumentar 10 % = `Custo * 1.10`.

```sql
UPDATE MANUTENCAO
SET Custo = Custo * 1.10
WHERE Tipo = 'Corretiva'
  AND TecnicoID IN (
        SELECT TecnicoID
        FROM TECNICO
        WHERE Especialidade = 'Transmissão'
  );
```

Estado da tabela depois de executar:

ManutencaoID	TecnicoID	Tipo	Custo
1	T01	Corretiva	132
2	T02	Preventiva	45
3	T03	Corretiva	150
4	T05	Preventiva	80
5	T01	Corretiva	231
6	T04	Corretiva	30
7	T02	Corretiva	104.5
8	T03	Preventiva	100

Só as intervenções corretivas dos técnicos T01 e T02 (Transmissão) subiram: 120→132, 210→231, 95→104,5 (o resto ficou igual; aqui vês as 8 linhas).

Solução alternativa (EXISTS correlacionado)

```sql
UPDATE MANUTENCAO
SET Custo = Custo * 1.10
WHERE Tipo = 'Corretiva'
  AND EXISTS (SELECT 1 FROM TECNICO AS T
              WHERE T.TecnicoID = MANUTENCAO.TecnicoID
                AND T.Especialidade = 'Transmissão');
```

Erros típicos. Escrever `WHERE Especialidade = 'Transmissão'` diretamente em MANUTENCAO (essa coluna não existe lá) · esquecer o `WHERE` (aumenta **tudo**) · `Custo * 0.10` ou `Custo + 10`.

Q3. Descrição, custo e nome do técnico responsável pelas intervenções com custo superior a 100, por custo decrescente.

Como pensar. Colunas de **duas tabelas** (Descricao/Custo em MANUTENCAO, Nome em TECNICO) → `JOIN` pela FK. "Superior a 100" → `> 100` (estrito). "Decrescente" → `ORDER BY … DESC`.

```sql
SELECT M.Descricao, M.Custo, T.Nome
FROM MANUTENCAO AS M
JOIN TECNICO AS T
  ON M.TecnicoID = T.TecnicoID
WHERE M.Custo > 100
ORDER BY M.Custo DESC;
```

Resultado nos dados de exemplo:

Descricao	Custo	Nome
Troca de cassete	210	Ana Sousa
Substituição de pastilhas	150	Carla Dias
Troca de corrente	120	Ana Sousa

Erros típicos. Esquecer o `ON` (produto cartesiano) · `TecnicoID` ambíguo sem alias · `>=` em vez de `>` · `ORDER BY` antes do `WHERE` (ordem das cláusulas).

Q4. Técnicos que ainda não realizaram qualquer intervenção.

Como pensar. "Nunca/nenhum" = **anti-junção**: o técnico para o qual *não existe* linha em MANUTENCAO. Um `INNER JOIN` não serve (só traz quem tem intervenções).

```
SELECT T.TecnicoID, T.Nome
FROM TECNICO AS T
WHERE NOT EXISTS (
        SELECT 1
        FROM MANUTENCAO AS M
        WHERE M.TecnicoID = T.TecnicoID
);
```

Resultado nos dados de exemplo:

TecnicoID	Nome
T06	Filipe Cruz

Alternativas aceites (a resolução oficial indica a do LEFT JOIN)

```
SELECT T.TecnicoID, T.Nome
FROM TECNICO AS T
LEFT JOIN MANUTENCAO AS M
  ON M.TecnicoID = T.TecnicoID
WHERE M.ManutencaoID IS NULL;
```

Resultado nos dados de exemplo:

TecnicoID	Nome
T06	Filipe Cruz

```
SELECT TecnicoID, Nome
FROM TECNICO
WHERE TecnicoID NOT IN (
        SELECT TecnicoID
        FROM MANUTENCAO
        WHERE TecnicoID IS NOT NULL
);
```

Armadilha do NOT IN. Se existir uma manutenção com `TecnicoID = NULL`, o `NOT IN` "sem proteção" **não devolve nada**. Com uma linha extra de teste (*manutenção sem técnico*):

```
-- Atenção: existe uma manutenção com TecnicoID = NULL
SELECT Nome FROM TECNICO
WHERE TecnicoID NOT IN (SELECT TecnicoID FROM MANUTENCAO);
```

Resultado nos dados de exemplo:

Nome
(0 linhas)

Por isso a variante acima inclui `WHERE TecnicoID IS NOT NULL`, e `NOT EXISTS` / `LEFT JOIN … IS NULL` são mais seguras.

Q5. Por especialidade: nº de técnicos e média de anos de experiência, só especialidades com pelo menos dois técnicos.

Como pensar. "Por especialidade" → `GROUP BY Especialidade`. "Número" → `COUNT(*)`; "média" → `AVG`. "Apenas as especialidades com pelo menos dois" é uma condição sobre o **grupo** → `HAVING`, não `WHERE`.

```sql
SELECT Especialidade,
       COUNT(*)             AS NumeroTecnicos,
       AVG(AnosExperiencia) AS MediaAnosExperiencia
FROM TECNICO
GROUP BY Especialidade
HAVING COUNT(*) >= 2;
```

Resultado nos dados de exemplo:

Especialidade	NumeroTecnicos	MediaAnosExperiencia
Transmissão	2	5.5
Travões	2	8.5

Erros típicos. `WHERE COUNT(*) >= 2` (erro: *misuse of aggregate*) · usar `> 2` em vez de `>= 2` · esquecer o `GROUP BY` · pôr no SELECT uma coluna não agregada que não está no GROUP BY.

2.7 Variações possíveis VARIAÇÃO

A professora pode usar a mesma técnica com outras tabelas, outros filtros, outro operador. Abaixo, uma bateria de pedidos **do mesmo género** que as perguntas do exame modelo — todos testados nos dados de exemplo. Treina escrevendo primeiro e só depois espreitando.

Variações de INSERT

Inserir duas categorias numa só instrução (5 · Pneus, 6 · Iluminação).

```sql
INSERT INTO CATEGORIA (CategoriaID, NomeCategoria)
VALUES (5, 'Pneus'),
       (6, 'Iluminação');
```

Estado da tabela depois de executar:

CategoriaID	NomeCategoria
1	Mecânica
2	Segurança
3	Estética
4	Elétrica
5	Pneus
6	Iluminação

Registar uma manutenção e associá-la à categoria 1 (duas tabelas, ordem importa: primeiro a MANUTENCAO, depois a tabela de associação).

```sql
INSERT INTO MANUTENCAO (ManutencaoID, Descricao, TecnicoID, Ano, Tipo, Custo)
VALUES (9, 'Troca de pneu', 'T05', 2026, 'Corretiva', 25.00);

INSERT INTO MANUTENCAO_CATEGORIA (ManutencaoID, CategoriaID)
VALUES (9, 1);
```

Se inverteres a ordem, a FK de MANUTENCAO_CATEGORIA falha (a manutenção ainda não existe). Mesmo raciocínio para apagar: primeiro as linhas "filhas".

Inserir um técnico sem certificação (omitir a coluna).

```sql
-- técnico sem certificação (a coluna omitida fica NULL)
INSERT INTO TECNICO (TecnicoID, Nome, AnosExperiencia, Especialidade)
VALUES ('T16', 'Hugo Neves', 0, 'Rodas');
```

Estado da tabela depois de executar:

TecnicoID	Nome	Certificacao	AnosExperiencia	Especialidade
T16	Hugo Neves	NULL	0	Rodas

Variações de UPDATE

Reduzir 5 % o custo das intervenções preventivas de 2025.

```sql
UPDATE MANUTENCAO
SET Custo = Custo * 0.95
WHERE Tipo = 'Preventiva'
  AND Ano = 2025;
```

Estado da tabela depois de executar:

ManutencaoID	Tipo	Ano	Custo
2	Preventiva	2025	42.75
4	Preventiva	2026	80
8	Preventiva	2026	100

Mudar a especialidade do técnico T04 para Transmissão e somar 1 ano de experiência (vários `SET`).

```sql
UPDATE TECNICO
SET Especialidade = 'Transmissão',
    AnosExperiencia = AnosExperiencia + 1
WHERE TecnicoID = 'T04';
```

Estado da tabela depois de executar:

TecnicoID	Nome	Certificacao	AnosExperiencia	Especialidade
T04	Diogo Faria	NULL	2	Transmissão

Somar 20 ao custo das intervenções de técnicos Shimano com pelo menos 10 anos de experiência (subconsulta com duas condições).

```sql
UPDATE MANUTENCAO
SET Custo = Custo + 20
WHERE TecnicoID IN (
        SELECT TecnicoID
        FROM TECNICO
        WHERE Certificacao = 'Shimano'
          AND AnosExperiencia >= 10
);
```

Estado da tabela depois de executar:

ManutencaoID	TecnicoID	Custo
3	T03	170
8	T03	120

Variações de DELETE

Apagar as manutenções de 2025 (e primeiro as suas associações a categorias, por causa da FK).

```sql
DELETE FROM MANUTENCAO_CATEGORIA
WHERE ManutencaoID IN (SELECT ManutencaoID FROM MANUTENCAO WHERE Ano = 2025);

DELETE FROM MANUTENCAO
WHERE Ano = 2025;
```

Estado da tabela depois de executar:

ManutencaoID	Ano
3	2026
4	2026
5	2026
7	2026
8	2026

Apagar as categorias que nunca foram usadas.

```sql
DELETE FROM CATEGORIA
WHERE CategoriaID NOT IN (
        SELECT CategoriaID
        FROM MANUTENCAO_CATEGORIA
);
```

Estado da tabela depois de executar:

CategoriaID	NomeCategoria
1	Mecânica
2	Segurança

Com dados de exemplo, desaparecem Estética e Elétrica. (Aqui `NOT IN` é seguro porque `CategoriaID` é parte da PK da tabela de associação e nunca é NULL.)

Variações de SELECT — filtros e ordenação

Nome e experiência dos técnicos Shimano com mais de 5 anos.

```sql
SELECT Nome, AnosExperiencia
FROM TECNICO
WHERE Certificacao = 'Shimano'
  AND AnosExperiencia > 5;
```

Resultado nos dados de exemplo:

Nome	AnosExperiencia
Ana Sousa	8
Carla Dias	12

Especialidades distintas, por ordem alfabética.

```sql
SELECT DISTINCT Especialidade
FROM TECNICO
ORDER BY Especialidade;
```

Resultado nos dados de exemplo:

Especialidade
Rodas
Suspensão
Transmissão
Travões

Manutenções cuja descrição começa por "Troca", em 2025–2026, dos tipos Corretiva ou Preventiva (`LIKE` + `BETWEEN` + `IN`).

```sql
SELECT ManutencaoID, Descricao, Ano, Custo
FROM MANUTENCAO
WHERE Descricao LIKE 'Troca%'
  AND Ano BETWEEN 2025 AND 2026
  AND Tipo IN ('Corretiva', 'Preventiva');
```

Resultado nos dados de exemplo:

ManutencaoID	Descricao	Ano	Custo
1	Troca de corrente	2025	120
5	Troca de cassete	2026	210
7	Troca de desviador	2026	95

Técnicos sem certificação registada.

```sql
SELECT Nome
FROM TECNICO
WHERE Certificacao IS NULL;
```

Resultado nos dados de exemplo:

Nome
Diogo Faria

As 3 intervenções mais caras (`LIMIT`).

```sql
SELECT Descricao, Custo
FROM MANUTENCAO
ORDER BY Custo DESC
LIMIT 3;
```

Resultado nos dados de exemplo:

Descricao	Custo
Troca de cassete	210
Substituição de pastilhas	150
Troca de corrente	120

Variações de SELECT — junções

Descrição, categoria e técnico das intervenções da categoria "Segurança" (4 tabelas, passa pela N:M).

```sql
SELECT M.Descricao, C.NomeCategoria, T.Nome AS Tecnico
FROM MANUTENCAO AS M
JOIN MANUTENCAO_CATEGORIA AS MC ON MC.ManutencaoID = M.ManutencaoID
JOIN CATEGORIA            AS C  ON C.CategoriaID   = MC.CategoriaID
JOIN TECNICO              AS T  ON T.TecnicoID     = M.TecnicoID
WHERE C.NomeCategoria = 'Segurança';
```

Resultado nos dados de exemplo:

Descricao	NomeCategoria	Tecnico
Substituição de pastilhas	Segurança	Carla Dias
Revisão de travões	Segurança	Eva Matos
Revisão geral	Segurança	Carla Dias

Quantas intervenções tem cada categoria utilizada?

```sql
SELECT C.NomeCategoria, COUNT(*) AS NumIntervencoes
FROM CATEGORIA AS C
JOIN MANUTENCAO_CATEGORIA AS MC ON MC.CategoriaID = C.CategoriaID
GROUP BY C.CategoriaID, C.NomeCategoria;
```

Resultado nos dados de exemplo:

NomeCategoria	NumIntervencoes
Mecânica	7
Segurança	3

Variações de SELECT — agregação

Custo total por técnico, só técnicos com total acima de 200, do maior para o menor.

```sql
SELECT T.Nome, SUM(M.Custo) AS CustoTotal
FROM TECNICO AS T
JOIN MANUTENCAO AS M ON M.TecnicoID = T.TecnicoID
GROUP BY T.TecnicoID, T.Nome
HAVING SUM(M.Custo) > 200
ORDER BY CustoTotal DESC;
```

Resultado nos dados de exemplo:

Nome	CustoTotal
Ana Sousa	330
Carla Dias	250

Número de intervenções por técnico, **incluindo os que têm zero** (`LEFT JOIN` + `COUNT(coluna da direita)`).

```sql
SELECT T.Nome, COUNT(M.ManutencaoID) AS NumIntervencoes
FROM TECNICO AS T
LEFT JOIN MANUTENCAO AS M ON M.TecnicoID = T.TecnicoID
GROUP BY T.TecnicoID, T.Nome
ORDER BY NumIntervencoes DESC, T.Nome;
```

Resultado nos dados de exemplo:

Nome	NumIntervencoes
Ana Sousa	2
Bruno Reis	2
Carla Dias	2
Diogo Faria	1
Eva Matos	1
Filipe Cruz	0

Com `COUNT(*)` o Filipe apareceria com 1 (a linha "vazia" do LEFT JOIN conta). `COUNT(M.ManutencaoID)` ignora NULL e dá o 0 correto.

Nº de intervenções e custo médio por ano e tipo (`GROUP BY` com duas colunas).

```sql
SELECT Ano, Tipo,
       COUNT(*)  AS NumIntervencoes,
       AVG(Custo) AS CustoMedio
FROM MANUTENCAO
GROUP BY Ano, Tipo
ORDER BY Ano, Tipo;
```

Resultado nos dados de exemplo:

Ano	Tipo	NumIntervencoes	CustoMedio
2025	Corretiva	2	75
2025	Preventiva	1	45
2026	Corretiva	3	151.67
2026	Preventiva	2	90

Custo total em 2026 por especialidade (junção + WHERE + GROUP BY + ORDER BY).

```sql
SELECT T.Especialidade, SUM(M.Custo) AS CustoTotal
FROM MANUTENCAO AS M
JOIN TECNICO AS T ON T.TecnicoID = M.TecnicoID
WHERE M.Ano = 2026
GROUP BY T.Especialidade
ORDER BY CustoTotal DESC;
```

Resultado nos dados de exemplo:

Especialidade	CustoTotal
Travões	330
Transmissão	305

Variações de SELECT — subconsultas

Intervenções com custo acima da média.

```sql
SELECT Descricao, Custo
FROM MANUTENCAO
WHERE Custo > (SELECT AVG(Custo) FROM MANUTENCAO);
```

Resultado nos dados de exemplo:

Descricao	Custo
Troca de corrente	120
Substituição de pastilhas	150
Troca de cassete	210

O(s) técnico(s) com mais anos de experiência.

```sql
SELECT Nome, AnosExperiencia
FROM TECNICO
WHERE AnosExperiencia = (SELECT MAX(AnosExperiencia) FROM TECNICO);
```

Resultado nos dados de exemplo:

Nome	AnosExperiencia
Carla Dias	12

Categorias nunca usadas (anti-junção na N:M).

```sql
SELECT C.NomeCategoria
FROM CATEGORIA AS C
WHERE NOT EXISTS (
        SELECT 1
        FROM MANUTENCAO_CATEGORIA AS MC
        WHERE MC.CategoriaID = C.CategoriaID
);
```

Resultado nos dados de exemplo:

NomeCategoria
Estética
Elétrica

Técnicos que fizeram intervenções corretivas e preventivas (dois `EXISTS`).

```sql
SELECT T.Nome
FROM TECNICO AS T
WHERE EXISTS (SELECT 1 FROM MANUTENCAO M
              WHERE M.TecnicoID = T.TecnicoID AND M.Tipo = 'Corretiva')
  AND EXISTS (SELECT 1 FROM MANUTENCAO M
              WHERE M.TecnicoID = T.TecnicoID AND M.Tipo = 'Preventiva');
```

Resultado nos dados de exemplo:

Nome
Bruno Reis
Carla Dias

Armadilha do COUNT.

```sql
SELECT COUNT(*) AS TodasLinhas, COUNT(Certificacao) AS ComCertificacao
FROM TECNICO;
```

Resultado nos dados de exemplo:

TodasLinhas	ComCertificacao
6	5

`COUNT(*)` conta todas as linhas; `COUNT(coluna)` só as que não são NULL.

Variações em cima do Grupo I (biblioteca)

O enunciado pode pedir SQL sobre o esquema do Grupo I. Com o esquema relacional da secção 1.7:

Título e editora de cada livro.

```sql
SELECT L.Titulo, E.Nome AS Editora
FROM LIVRO AS L
JOIN EDITORA AS E ON E.EditoraID = L.EditoraID
ORDER BY L.Titulo;
```

Resultado nos dados de exemplo:

Titulo	Editora
1984	Penguin
Dom Quixote	Planeta
Mensagem	Porto Editora
Os Maias	Porto Editora

Leitores que nunca pediram um empréstimo.

```sql
SELECT Nome
FROM LEITOR AS L
WHERE NOT EXISTS (SELECT 1 FROM EMPRESTIMO E WHERE E.NumeroLeitor = L.NumeroLeitor);
```

Resultado nos dados de exemplo:

Nome
Tiago Pinto

Títulos devolvidos "Danificado" e a data de levantamento (passa pela tabela N:M).

```sql
SELECT LI.Titulo, EL.EstadoDevolucao, EM.DataLevantamento
FROM EMPRESTIMO_LIVRO AS EL
JOIN LIVRO      AS LI ON LI.ISBN         = EL.ISBN
JOIN EMPRESTIMO AS EM ON EM.EmprestimoID = EL.EmprestimoID
WHERE EL.EstadoDevolucao = 'Danificado';
```

Resultado nos dados de exemplo:

Titulo	EstadoDevolucao	DataLevantamento
1984	Danificado	2026-03-01

Editoras com pelo menos dois livros.

```sql
SELECT E.Nome, COUNT(*) AS NumLivros
FROM EDITORA AS E
JOIN LIVRO AS L ON L.EditoraID = E.EditoraID
GROUP BY E.EditoraID, E.Nome
HAVING COUNT(*) >= 2;
```

Resultado nos dados de exemplo:

Nome	NumLivros
Porto Editora	2

2.8 Exercícios das aulas (universidade e livraria)

Os slides trazem dois cenários com tarefas em 3 partes: **CRUD**, **filtros** e **junções/agregação**.

Universidade — tabelas `students` (student_id, name, major, enrollment_year) e `grades` (grade_id, student_id, course_name, score)

```sql
-- 1. Create: nova aluna
INSERT INTO students (name, major, enrollment_year)
VALUES ('Alice Smith', 'Computer Science', 2024);
-- 2. Update: mudou de curso
UPDATE students SET major = 'Data Science' WHERE name = 'Alice Smith';
-- 3. Delete: curso cancelado
DELETE FROM grades WHERE course_name = 'Intro to Basket Weaving';
```

Filtros — majors *distintos* de alunos inscritos entre 2018 e 2022, com nome a começar por J, em Physics/Mathematics/Engineering:

```sql
SELECT DISTINCT major
FROM students
WHERE enrollment_year BETWEEN 2018 AND 2022
  AND name LIKE 'J%'
  AND major IN ('Physics', 'Mathematics', 'Engineering');
```

Resultado nos dados de exemplo:

major
Physics
Mathematics
Engineering

Junção + agregação — os **2 majors** com maior média de notas, só com média estritamente superior a 85: *JOIN → GROUP BY major → HAVING → ORDER BY → LIMIT*:

```sql
SELECT s.major, AVG(g.score) AS average_score
FROM students s
JOIN grades g ON s.student_id = g.student_id
GROUP BY s.major
HAVING AVG(g.score) > 85
ORDER BY average_score DESC
LIMIT 2;
```

Resultado nos dados de exemplo:

major	average_score
Mathematics	91.5
Physics	90.33

Livraria — tabelas `books` (book_id, title, author, genre, price) e `sales` (sale_id, book_id, quantity, sale_date)

```sql
INSERT INTO books (title, author, genre, price) VALUES ('Dune', 'Frank Herbert', 'Sci-Fi', 14.99);
UPDATE books SET price = 9.99 WHERE title = '1984';
DELETE FROM books WHERE title = 'Encyclopedia Vol 1';
```

Top 3 títulos únicos mais caros, género em (Mystery, Sci-Fi), preço entre 10 e 20, autor a começar por "A", por preço decrescente:

```sql
SELECT DISTINCT title, price
FROM books
WHERE genre IN ('Mystery', 'Sci-Fi')
  AND price BETWEEN 10.00 AND 20.00
  AND author LIKE 'A%'
ORDER BY price DESC
LIMIT 3;
```

Resultado nos dados de exemplo:

title	price
Atlas Dark	19
Gone Girl	15.5

Receita total por género (`SUM(price * quantity)`), só géneros com mais de 250:

```sql
SELECT b.genre, SUM(b.price * s.quantity) AS total_revenue
FROM books b
JOIN sales s ON b.book_id = s.book_id
GROUP BY b.genre
HAVING SUM(b.price * s.quantity) > 250;
```

Resultado nos dados de exemplo:

genre	total_revenue
Mystery	305.5
Sci-Fi	429.7

Inconsistências nos slides — não copies cegamente. (1) Na slide "Solutions: Part 2" da livraria o enunciado diz autor a começar por **A** mas a solução usa `LIKE 'F%'`. (2) Na "Part 3" o enunciado diz **250** e a solução usa `> 100`. (3) A nota "*LIKE '>%'*" na universidade é gralha de `'J%'`. No exame, **segue sempre o enunciado**.
(4) A solução usa o alias `total_revenue` no `HAVING`: funciona em SQLite/MySQL, mas **não é SQL padrão** (SQL Server/Oracle dão erro). Repetir a expressão — `HAVING SUM(price*quantity) > 250` — funciona em todo o lado.

2.9 Checklist rápida de SQL

Antes de entregares...	Verifica
Texto e números	Texto entre **aspas simples**, números sem aspas; acentos iguais ao enunciado (*Travões*, *Transmissão*).
UPDATE / DELETE	Tem `WHERE`? Filtra o que o enunciado pede?
JOIN	Tem `ON` com FK = PK? Colunas ambíguas qualificadas com alias?
"Nenhum / nunca / sem..."	Anti-junção: `NOT EXISTS` ou `LEFT JOIN … IS NULL`.
"Por X / para cada X"	`GROUP BY X`; tudo o que não é agregado no SELECT está no GROUP BY.
"Apenas os que têm pelo menos / mais de..."	`HAVING` (grupo) — e "pelo menos 2" é `>= 2`, "mais de 2" é `> 2`.
"Ordenadas por ... decrescente"	`ORDER BY … DESC` no fim; "os 3 primeiros" → `LIMIT 3`.
Pontuação	Vírgulas entre colunas, nenhuma antes do `FROM`; `;` no fim.

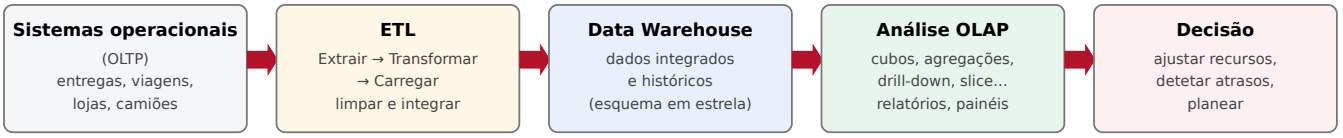
3 · Módulo 3

Inteligência para Negócios: OLTP, OLAP, Data Warehouse, ETL e modelo dimensional

> **Peso no exame:** 5 valores em 20. Matéria mais "de conceito": vale a pena dominar as definições e ser capaz de **desenhar um modelo dimensional simples**.

3.1 Dos dados à decisão

- **Inteligência para Negócios (BI)**: métodos e tecnologias que transformam dados em informação útil para apoiar decisões — *recolher e integrar*, *organizar e analisar*, *comunicar* (relatórios, indicadores, painéis).
- **Cadeia: Dados** (registos: data, loja, viagem) → **Informação** (nº de entregas por mês) → **Conhecimento** (que meses/centros têm mais volume) → **Decisão** (ajustar recursos). *Os dados, por si só, não garantem uma boa decisão.*
- **Começar pelas perguntas de negócio**: "Quantas entregas por mês?", "Que centros realizam mais entregas?". Antes de analisar, define: a pergunta, os dados necessários e a **medida** que responde.
- **Processo de negócio** = conjunto de atividades relacionadas que produz um resultado (preparar encomenda → organizar viagem → transportar → entregar). Cada atividade **gera dados**; estes ficam em sistemas diferentes, com formatos distintos, erros e duplicados → têm de ser **integrados e preparados**.
- **Big Data**: dados cuja dimensão, rapidez de geração ou diversidade tornam o tratamento mais exigente — os 3 V: **Volume**, **Velocidade**, **Variedade**.



Da operação à decisão: onde entram OLTP, ETL, Data Warehouse e OLAP.

3.2 OLTP vs OLAP · Data Warehouse · ETL

	OLTP (*Online Transaction Processing*)	**OLAP (*Online Analytical Processing*)**
Objetivo	Registar as operações diárias	Analisar dados e apoiar decisões
Operações	Inserir e atualizar registos	Agregar e comparar dados
Dados	Sobretudo atuais e detalhados	Integrados e históricos
Consultas	Geralmente curtas (muitas transações curtas)	Analíticas, muitas vezes complexas
Utilizadores	Aplicações e equipas operacionais	Analistas e decisores
Exemplo	Registar uma entrega	Comparar entregas por ano e por centro

> **Em resumo (slide):** OLTP regista *o que acontece*; OLAP ajuda a perceber *o que aconteceu*.

Porque não analisar diretamente no sistema operacional? As BD operacionais são feitas para registar/atualizar; uma análise histórica pode exigir muitos registos e juntar várias tabelas, pode não existir todo o histórico, e consultas demoradas **prejudicam o desempenho das operações diárias**.

- **Data Warehouse (DW)**: repositório que **integra** dados de diferentes fontes e os organiza para análise; conserva **informação histórica**; permite comparar períodos, locais e tipos de operação; facilita relatórios e indicadores.

- **ETL** = **E**xtração (recolher dos sistemas de origem), **T**ransformação (validar, limpar, uniformizar), **C**arregamento (guardar no DW). Ex. de transformação: uniformizar formatos de data, corrigir códigos de loja inconsistentes, tratar duplicados.
- **Arquiteturas OLAP**: **ROLAP** (tabelas relacionais e consultas SQL), **MOLAP** (estruturas multidimensionais, podendo pré-calcular agregações), **HOLAP** (combina as duas). Escolha depende do volume, do desempenho pretendido e da tecnologia.

3.3 O modelo dimensional

Conceito	Definição	Exemplo (entregas)
Facto	O acontecimento que queremos analisar	Uma etapa de entrega / uma entrega
Medida	Valor que queremos quantificar	Peso transportado, volume, custo, nº de entregas
Dimensão	Perspetiva/contexto de análise do facto; atributos descritivos que permitem filtrar e agrupar	Data, loja, centro, camião
Granularidade	**Nível de detalhe de cada registo** da tabela de factos	"Uma etapa de entrega numa viagem" → cada linha é uma etapa
Hierarquia	Organiza atributos do mais detalhado para o mais geral numa dimensão	dia → mês → trimestre/ano · loja → cidade → distrito · produto → categoria

> **Granularidade primeiro.** Define-a *antes* de escolher as medidas; tem de ser clara e consistente em toda a tabela. Uma definição incorreta leva à **contagem duplicada** de entregas, peso ou quilómetros. Cada linha da tabela de factos tem de corresponder ao evento definido.

Tabela de factos vs tabelas de dimensão

Tabela de factos: regista eventos ao nível da granularidade; contém **medidas**; inclui **chaves estrangeiras** para as dimensões (ex.: `DataKey` → DimData, `LojaKey` → DimLoja).

Tabelas de dimensão: descrevem o contexto do evento; contêm **atributos descritivos**; permitem filtrar e agrupar os factos. Cada dimensão tem a sua **PK**, referenciada pela FK da tabela de factos.

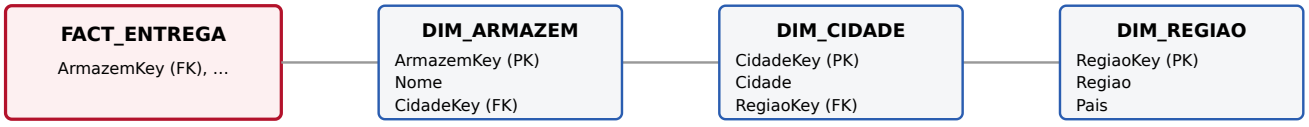
Tipos de medidas (aditividade)

Tipo	Regra	Exemplo
Aditiva	Pode ser **somada em todas** as dimensões	Peso entregue (se cada encomenda for contabilizada uma só vez); nº de entregas; custo
Semi-aditiva	Somável em **algumas** dimensões, não em todas	Quilómetros acumulados de um camião ao longo do tempo (somas entre camiões ok; somar no tempo não)
Não aditiva	**Não se soma**; recalcula-se a partir dos valores que lhe dão origem	% de entregas pontuais, médias, rácios (custo por km)

A resolução do exame modelo repete a ideia: "uma média ou uma percentagem não é, em geral, aditiva: deve ser recalculada a partir dos valores que lhe dão origem". Uma medida só é fiável se estiver definida de forma consistente com a granularidade.

Esquemas: estrela, floco de neve, galáxia

Esquema	Estrutura	Característica principal
Star (estrela)	Uma tabela de factos ligada **diretamente** às dimensões	Simples de compreender e consultar
Snowflake (floco de neve)	Dimensões **divididas em tabelas relacionadas**	Reduz redundância, mas pode exigir mais junções
Galaxy (constelação)	**Várias tabelas de factos** que podem partilhar dimensões	Permite analisar vários processos de negócio; estrutura mais complexa



Snowflake: a dimensão Armazém é normalizada em várias tabelas (mais junções, menos redundância).

Galaxy: p.ex. analisar "entregas" e "custos de transporte" em duas tabelas de factos que partilham as dimensões Data, Centro e Camião.

3.4 Operações OLAP

As operações permitem **resumir, detalhar, filtrar e reorganizar** a análise sobre o **cubo** (medidas analisadas por várias dimensões; cada combinação de valores das dimensões corresponde a uma célula/resultado; podemos analisar uma dimensão isolada ou combinar várias).

Operação	O que faz	Exemplo	Truque
Roll-up	Passa para um nível **mais geral** (resumir)	Entregas por **mês** → entregas por **ano**	menos detalhe
Drill-down	Passa para um nível **mais específico** (detalhar)	Entregas por **ano** → por **mês** (ou por dia)	mais detalhe
Slice	**Fixa um valor numa dimensão**	Só as entregas de **2026**	1 valor, 1 dimensão
Dice	Seleciona dados com **vários critérios** (sub-cubo)	Lojas de Braga **e** Guimarães **e** camiões pesados	combinação de critérios
Pivot	**Troca a disposição** das dimensões (linhas ↔ colunas); não altera os dados	Anos nas linhas e tipos de camião nas colunas → trocar	só muda a vista

Ver as operações em números

Dados de exemplo do esquema em estrela da secção 3.5 (8 entregas). Cada tabela abaixo é o resultado **real** da consulta SQL indicada.

Roll-up (nível Ano)

Resultado nos dados de exemplo:

Ano	NumEntregas	PesoTotalKg
2025	2	4300
2026	6	23150

Drill-down (nível Ano → Mês)

Resultado nos dados de exemplo:

Ano	Mes	NumEntregas
2025	11	1
2025	12	1
2026	1	2
2026	2	2
2026	3	2

Slice: Ano = 2026 (por cidade do armazém)

Resultado nos dados de exemplo:

Cidade	NumEntregas
Aveiro	2
Lisboa	1
Porto	3

Dice: Porto e Aveiro **e** veículos pesados

Resultado nos dados de exemplo:

Cidade	TipoVeiculo	NumEntregas
Aveiro	Pesado	2
Porto	Pesado	2

Antes do pivot (Mês nas linhas, tipo de veículo nas colunas)

Resultado nos dados de exemplo:

Mes	Ligeiro	Pesado
1	1	1
2	0	2
3	1	1

Depois do pivot (tipo nas linhas, mês nas colunas)

Tipo	Mes 1	Mes 2	Mes 3
Ligeiro	1	0	1
Pesado	1	2	1

Mesmos números, nova disposição.

> **Dica slice vs dice.** **Slice** = *um valor* numa dimensão. **Dice** = combinação de critérios (vários valores e/ou mais de uma dimensão). "Armazéns do Porto **e** de Aveiro, apenas veículos pesados" → **dice** (dois valores + duas dimensões). "Apenas 2026" → **slice**.

As operações em SQL (esquema em estrela)

O padrão é sempre: `FROM` a tabela de factos `JOIN` as dimensões necessárias `GROUP BY` os atributos da dimensão pelos quais queres analisar, com `SUM/COUNT/AVG` sobre as medidas.

```sql
SELECT t.Ano, COUNT(*) AS NumEntregas, SUM(f.QuantidadeKg) AS PesoTotalKg
FROM FACT_ENTREGA AS f
JOIN DIM_TEMPO    AS t ON f.TempoKey = t.TempoKey
GROUP BY t.Ano
ORDER BY t.Ano;
```

Resultado nos dados de exemplo:

Ano	NumEntregas	PesoTotalKg
2025	2	4300
2026	6	23150

Drill-down: acrescenta o nível mais fino ao `SELECT` / `GROUP BY` (`t.Ano, t.Mes`). **Roll-up**: retira-o. **Slice**: acrescenta `WHERE t.Ano = 2026`. **Dice**: `WHERE a.Cidade IN ('Porto','Aveiro') AND v.TipoVeiculo = 'Pesado'`. **Pivot**: um `SUM(CASE WHEN … THEN 1 ELSE 0 END)` por coluna.

```sql
SELECT a.Cidade, v.TipoVeiculo, COUNT(*) AS NumEntregas
FROM FACT_ENTREGA AS f
JOIN DIM_ARMAZEM AS a ON f.ArmazemKey = a.ArmazemKey
JOIN DIM_VEICULO AS v ON f.VeiculoKey = v.VeiculoKey
WHERE a.Cidade IN ('Porto', 'Aveiro')
  AND v.TipoVeiculo = 'Pesado'
GROUP BY a.Cidade, v.TipoVeiculo;
```

```sql
SELECT t.Mes,
       SUM(CASE WHEN v.TipoVeiculo = 'Ligeiro' THEN 1 ELSE 0 END) AS Ligeiro,
       SUM(CASE WHEN v.TipoVeiculo = 'Pesado'  THEN 1 ELSE 0 END) AS Pesado
FROM FACT_ENTREGA AS f
JOIN DIM_TEMPO    AS t ON f.TempoKey    = t.TempoKey
JOIN DIM_VEICULO  AS v ON f.VeiculoKey = v.VeiculoKey
WHERE t.Ano = 2026
GROUP BY t.Mes
ORDER BY t.Mes;
```

Outras consultas analíticas (slides)

- **Top-N** (os 5 distritos com maior peso transportado) → `ORDER BY … DESC LIMIT N`.
- **Por limiar** (lojas com mais de 100 entregas) → `HAVING COUNT(*) > 100`.
- **Tendência** (evolução mensal) → agrupar por mês, ordenar por tempo.
- **Comparação temporal** (2025 vs 2026) → agrupar por ano ou duas colunas condicionais.

```sql
SELECT a.Cidade, SUM(f.QuantidadeKg) AS PesoTotalKg
FROM FACT_ENTREGA AS f
JOIN DIM_ARMAZEM  AS a ON f.ArmazemKey = a.ArmazemKey
GROUP BY a.Cidade
ORDER BY PesoTotalKg DESC
LIMIT 2;
```

Resultado nos dados de exemplo:

Cidade	PesoTotalKg
Porto	11550
Lisboa	8000

```sql
SELECT a.Cidade,
       SUM(CASE WHEN t.Ano = 2025 THEN 1 ELSE 0 END) AS Entregas2025,
       SUM(CASE WHEN t.Ano = 2026 THEN 1 ELSE 0 END) AS Entregas2026
FROM FACT_ENTREGA AS f
JOIN DIM_TEMPO    AS t ON f.TempoKey    = t.TempoKey
JOIN DIM_ARMAZEM  AS a ON f.ArmazemKey = a.ArmazemKey
GROUP BY a.Cidade;
```

Resultado nos dados de exemplo:

Cidade	Entregas2025	Entregas2026
Aveiro	1	2
Lisboa	0	1
Porto	1	3

Desempenho: índices adequados às consultas · pré-cálculo de agregações frequentes · particionamento (p.ex. por data) · otimização de junções e consultas. A escolha depende do volume, das consultas e da frequência de atualização.

3.5 Exame modelo · Grupo III resolvido EXAME MODELO

> **Contexto.** Uma empresa de distribuição alimentar analisa as entregas aos clientes empresariais; a direção quer comparar resultados por **período temporal, armazém, zona geográfica e tipo de veículo**.

Q1 — Verdadeiro ou falso (com justificação breve)

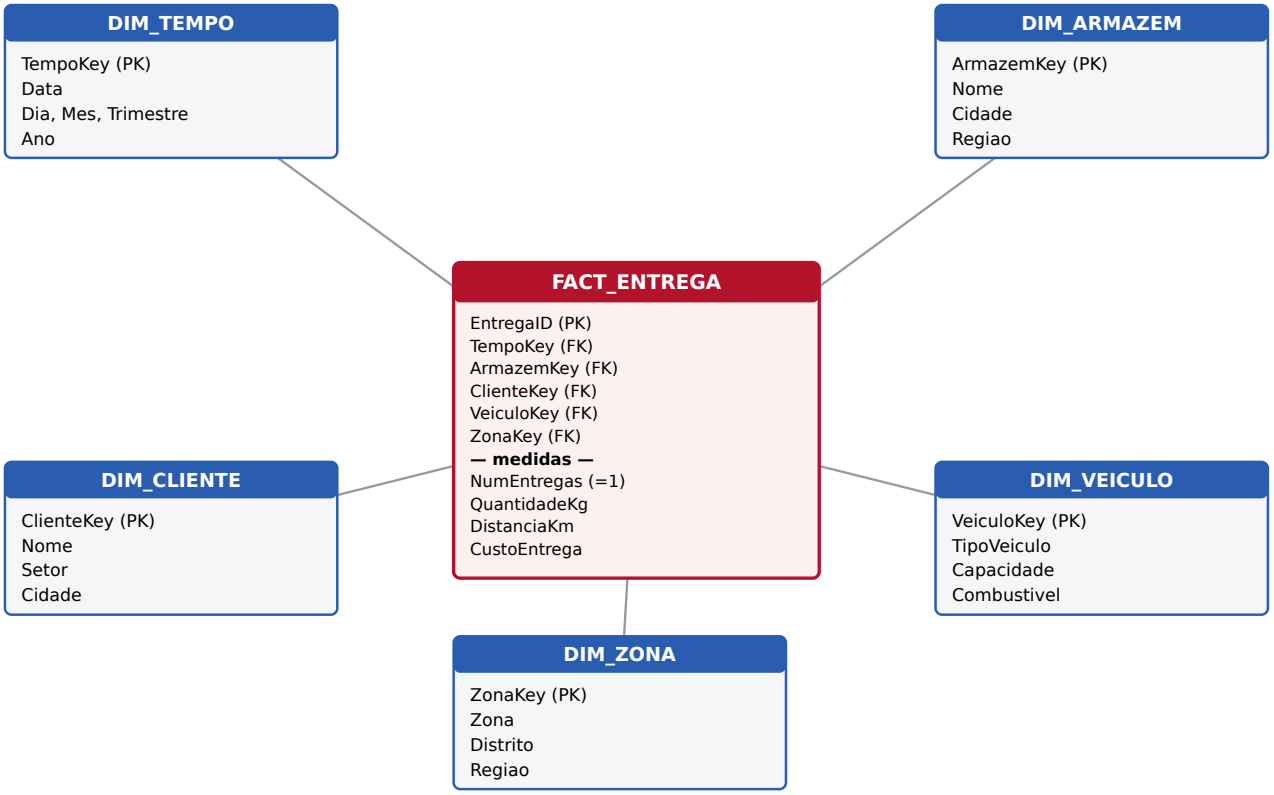
Afirmação	Resposta	Justificação (modelo)
a) Um sistema **OLTP** é normalmente utilizado para registar operações do dia a dia.	**Verdadeiro**	Suporta as operações correntes (registar encomendas, atualizar dados operacionais).
b) Um **Data Warehouse** integra dados para apoiar a análise e a tomada de decisão.	**Verdadeiro**	Integra e organiza dados, frequentemente de várias fontes, para análise e decisão.
c) O processo **ETL** inclui extração, transformação e carregamento dos dados.	**Verdadeiro**	ETL = *Extract, Transform, Load*.
d) Um sistema **OLAP** destina-se sobretudo ao registo de transações operacionais curtas.	**Falso**	OLAP destina-se à **análise multidimensional** e exploração de informação para apoio à decisão; o registo de transações curtas é o OLTP.

Como responder: "Verdadeiro/Falso" + **uma frase** que mostre o conceito certo. Nas falsas, diz o que seria verdadeiro (a troca OLTP↔OLAP é o clássico).

Q2 — Modelo dimensional

Como pensar (4 passos):

1. **Processo de negócio / facto**: as entregas aos clientes.
2. **Granularidade** (primeiro!): **um registo por entrega realizada**.
3. **Dimensões** — repara que o enunciado já as dá: "*por período temporal, armazém, zona geográfica e tipo de veículo*" → Tempo, Armazém, Zona, Veículo (a resolução acrescenta Cliente, pois são "clientes empresariais").
4. **Medidas**: valores numéricos coerentes com a granularidade: nº de entregas (1 por linha), quantidade entregue, distância percorrida, custo.



Esquema em estrela para as entregas (granularidade: uma linha por entrega realizada).

Tabela	Atributos
FACT_ENTREGA	FK para cada dimensão (TempoKey, ArmazemKey, ClienteKey, VeiculoKey, ZonaKey) + medidas: **NumEntregas** (indicador 1), **QuantidadeKg**, **DistanciaKm**, **CustoEntrega**
DIM_TEMPO	data, dia, mês, trimestre, ano (hierarquia dia → mês → trimestre → ano)
DIM_ARMAZEM	identificador, nome, cidade, região
DIM_CLIENTE	identificador, nome, setor de atividade, cidade
DIM_VEICULO	identificador, tipo de veículo, capacidade, combustível
DIM_ZONA	identificador, zona geográfica, distrito, região

Aditividade das medidas aqui. NumEntregas, quantidade, distância e custo são **aditivas** (registadas por entrega, somam-se por tempo, armazém, zona…). Já um **custo por km** ou uma **percentagem de entregas pontuais** são **não aditivas**: calcula-se `SUM(custo)/SUM(distância)` no nível pretendido, nunca somando rácios.

```sql
-- custo médio por km: recalcular a partir das somas (não somar/médias de rácios)
SELECT a.Cidade,
       SUM(f.CustoEntrega) / SUM(f.DistanciaKm) AS CustoPorKm
FROM FACT_ENTREGA AS f
JOIN DIM_ARMAZEM  AS a ON f.ArmazemKey = a.ArmazemKey
GROUP BY a.Cidade;
```

Resultado nos dados de exemplo:

Cidade	CustoPorKm
Aveiro	2.23
Lisboa	4.5
Porto	3.60

Erros típicos. (1) Não dizer a granularidade, ou dizê-la mal (“por armazém” — isso é uma dimensão, não o grão). (2) Pôr medidas como dimensões (ou o contrário: “tipo de veículo” é dimensão). (3) Menos de 4 dimensões ou sem atributos. (4) Esquecer as **chaves** (PK nas dimensões, FK na tabela de factos). (5) Medidas incompatíveis com o grão (ex.: “média de entregas por mês” numa tabela com uma linha por entrega).

Q3 — Operações OLAP

Situação	Operação	Porquê
a) Passar da análise **anual** do nº de entregas para uma **mensal**	**Drill-down**	Mais detalhe na hierarquia do tempo (ano → mês).
b) Consultar **apenas** as entregas de **2026**	**Slice**	Fixa **um valor** (2026) numa dimensão (Tempo).
c) Armazéns do **Porto e Aveiro**, só veículos **pesados**	**Dice**	Combina critérios em duas dimensões (Armazém e Veículo), com mais de um valor.
d) **Trocar linhas e colunas** de uma tabela (mês × tipo de veículo)	**Pivot**	Só muda a disposição das dimensões na apresentação.

3.6 Variações possíveis VARIAÇÃO

O que pode mudar: outro contexto (retalho, hospital, cinema, manutenção...); V/F sobre outras afirmações da matéria (aditividade, granularidade, esquemas, ROLAP/MOLAP, 3 V); pedir **modelo em floco de neve** ou com **hierarquias** explícitas; pedir a **operação OLAP certa** (incluindo **roll-up**) para outras situações; pedir **uma consulta SQL** sobre o modelo dimensional; perguntar **porque não usar o OLTP** para análise.

V1 — Banco de afirmações verdadeiro/falso (com respostas)

Afirmação	V/F	Porquê
Um Data Warehouse conserva apenas os dados atuais.	**F**	Conserva **informação histórica** integrada.
A granularidade deve ser definida depois de escolher as medidas.	**F**	Define-se **antes**; as medidas têm de ser coerentes com ela.
A tabela de factos contém medidas e chaves estrangeiras para as dimensões.	**V**	É a sua definição.
As percentagens de entregas pontuais podem ser somadas entre armazéns.	**F**	Não aditiva: recalcula-se a partir dos valores de origem.
Roll-up é passar para um nível mais detalhado.	**F**	Roll-up = menos detalhe; **drill-down** = mais detalhe.
No esquema em estrela as dimensões ligam diretamente à tabela de factos.	**V**	Simples de consultar; sem dimensões intermédias.
O esquema snowflake reduz redundância mas pode exigir mais junções.	**V**	Dimensões normalizadas em várias tabelas.
O ETL consiste apenas em extrair os dados dos sistemas de origem.	**F**	Extrair, **transformar** (limpar/uniformizar) e **carregar**.
Um slice fixa vários valores em várias dimensões.	**F**	Slice fixa **um valor** numa dimensão; a combinação é o dice.
Um pivot altera os dados guardados no Data Warehouse.	**F**	Só muda a forma de visualizar o resultado.
MOLAP usa estruturas multidimensionais e pode pré-calcular agregações.	**V**	ROLAP usa tabelas relacionais/SQL; HOLAP combina.
Big Data refere-se apenas ao volume de dados.	**F**	Também a velocidade e a variedade (3 V).
As consultas OLTP são tipicamente analíticas e complexas.	**F**	OLTP: transações geralmente curtas; analíticas = OLAP.
Correr análises pesadas sobre o sistema operacional pode prejudicar as operações diárias.	**V**	Motivo para existir um ambiente preparado para análise (DW).
Um sistema OLTP é o destino do carregamento do ETL.	**F**	O destino é o Data Warehouse; o OLTP é **origem**.

V2 — Classificar medidas (aditiva / semi-aditiva / não aditiva)

Medida	Classificação	Raciocínio
Peso entregue por encomenda (cada encomenda contada uma vez)	Aditiva	Soma-se por tempo, armazém, veículo…
Nº de entregas realizadas	Aditiva	Contagem somável em todas as dimensões.
Quilómetros acumulados do camião (conta-quilómetros)	Semi-aditiva	Somas entre camiões fazem sentido; somar leituras ao longo do tempo não.
% de entregas pontuais	Não aditiva	Recalcular: entregas pontuais ÷ entregas totais.
Custo médio por entrega	Não aditiva	Média: recalcular = custo total ÷ nº de entregas.
Custo por km	Não aditiva	`SUM(custo)/SUM(km)`.
Saldo de stock no fim do dia	Semi-aditiva	Soma-se por armazém, não ao longo dos dias.

V3 — Identificar a operação OLAP

Situação	Operação
Passar das vendas por **dia** para vendas por **trimestre**	**Roll-up** (menos detalhe)
Ver só as entregas do armazém **de Aveiro**	**Slice** (um valor, uma dimensão)
Ver só as entregas de **2025 e 2026** dos veículos **elétricos** nas zonas **Norte** e **Centro**	**Dice** (vários critérios)
Passar de "por cidade" para "por **loja** dentro de cada cidade"	**Drill-down**
Mostrar zonas nas colunas e anos nas linhas em vez do contrário	**Pivot**
Dos **distritos** para o total do **país**	**Roll-up**

V4 — Outro contexto: manutenção de bicicletas (liga o Grupo III ao Grupo II)

> **Enunciado.** A empresa de aluguer de bicicletas quer analisar os custos de manutenção por **ano**, **técnico**, **especialidade**, **categoria de serviço** e **tipo de intervenção**. Proponha um modelo dimensional.

- **Processo:** intervenções de manutenção. **Grão:** *uma intervenção de manutenção* (uma linha por `ManutencaoID`).
- **Medidas: Custo** (aditiva), **NumIntervencoes** (=1, aditiva). Custo médio = SUM(Custo)/SUM(NumIntervencoes) (não aditiva, recalcular).
- **Dimensões:** DIM_TEMPO (ano; se houver data: mês, trimestre) · DIM_TECNICO (nome, certificação, anos de experiência, especialidade) · DIM_TIPO (corretiva/preventiva) · DIM_CATEGORIA (nome da categoria).
- **Atenção ao grão!** Uma intervenção pode ter **várias categorias** (tabela N:M no relacional). Se a tabela de factos tiver uma linha por intervenção *e por categoria*, o custo **conta-se duas vezes** para quem tem duas categorias — viola a ideia "cada linha = o evento definido". Soluções: (a) grão "intervenção × categoria" e **não somar** o custo por intervenção em análises sem categoria; (b) tabela de *bridge*; (c) categoria principal. O ponto a mostrar no exame é **perceber o risco da dupla contagem**.

O mesmo exemplo em **snowflake**: DIM_TECNICO → DIM_ESPECIALIDADE (a especialidade passa para a sua própria tabela, ligada por FK).

V5 — Esboços rápidos para outros contextos

Contexto	Grão	Medidas	Dimensões (atributos)
Retalho	uma linha de venda (produto em cada talão)	quantidade, valor, desconto	Tempo (dia, mês, ano) · Loja (cidade, distrito) · Produto (categoria) · Cliente · Promoção
Hospital	uma consulta	duração, custo, nº consultas	Tempo · Médico (especialidade) · Paciente (idade, plano) · Serviço/Ala · Diagnóstico
Cinema	um bilhete vendido	nº bilhetes (1), receita	Tempo · Cinema (cidade) · Filme (género) · Sessão (horário, tipo de sala) · Tipo de bilhete
Transportes (aulas)	uma etapa de entrega numa viagem	peso, volume, km, nº etapas	Data · Loja · Centro · Camião (tipo) · Viagem
Biblioteca (Gr. I)	um livro num empréstimo	nº livros emprestados, dias de atraso	Tempo · Livro (editora, ano) · Leitor · Estado devolução

V6 — Pergunta de negócio → consulta (liga SQL e BI)

"Que armazéns tiveram mais de 2 entregas?" → junção facto-dimensão + `HAVING`:

```sql
SELECT a.Cidade, COUNT(*) AS NumEntregas
FROM FACT_ENTREGA AS f
JOIN DIM_ARMAZEM  AS a ON f.ArmazemKey = a.ArmazemKey
GROUP BY a.Cidade
HAVING COUNT(*) > 2;
```

Resultado nos dados de exemplo:

Cidade	NumEntregas
Aveiro	3
Porto	4

Truque de tradução: as palavras "*por* X" dão o `GROUP BY` (e a dimensão); "*quantas/quanto*" dão a medida e a agregação; "*só* em 2026" dá o `WHERE` (slice); "*mais de* N" dá o `HAVING`.

4 · Exame de treino

Mesma estrutura do exame modelo, outros domínios — faz em 90 minutos, sem consultar, e só depois confere as soluções

> **Instruções.** Duração 90 min · cotação sugerida: **Grupo I = 7,5 · Grupo II = 7,5 · Grupo III = 5** (15 + 5, como no email). Usa só a tua folha A4 manuscrita. Os dados de exemplo da Loja Online (Grupo II) estão no anexo B para quem quiser testar as consultas em SQLite — no exame real não terás os dados, só o esquema.

Grupo I (7,5 val)

Uma escola de música pretende informatizar a gestão de alunos, professores, instrumentos e aulas. Regras de negócio:

- Cada **aluno** tem um número de aluno, um nome, uma data de nascimento e um ou mais contactos telefónicos.
- Cada **professor** tem um identificador, um nome e um endereço de correio eletrónico.
- Cada **instrumento** tem um código e um nome. Um professor leciona **pelo menos um** instrumento e pode lecionar vários; cada instrumento pode ser lecionado por vários professores (ou por nenhum, se estiver temporariamente sem professor).
- Cada **aula** tem um número, uma data/hora e uma sala. É dada por **um único professor** e incide sobre **um único instrumento**. Uma aula tem sempre **pelo menos um aluno** (aula individual ou de grupo) e um aluno pode frequentar várias aulas. Para cada aluno numa aula regista-se a **classificação**, que pode ainda não existir.

1. Construa o modelo Entidade–Relacionamento: entidades, principais atributos, chaves e cardinalidades (mín, máx). *(3,5 val)*
2. Apresente o esquema relacional correspondente, identificando chaves primárias e estrangeiras. *(2,5 val)*
3. Considere PROFESSOR(ProfID, Nome, Email) com os tuplos (1, Ana, ana@m.pt) e (2, Rui, rui@m.pt), e AULA(AulaID, DataHora, Sala, *ProfID*) com ProfID a referenciar PROFESSOR e existindo uma aula do professor 1. Indique, justificando, se cada operação viola alguma restrição de integridade (e qual): *(1,5 val)*
   a) inserir uma aula com ProfID = 3 b) inserir um professor com ProfID = NULL c) apagar o professor 1 d) inserir um professor (3, Eva, ana@m.pt) sabendo que o Email é chave candidata.

Grupo II (7,5 val)

Uma loja online tem as relações seguintes (chaves primárias sublinhadas; as FK estão indicadas em itálico):

- CLIENTE(ClienteID, Nome, Cidade, DataRegisto)
- PRODUTO(ProdutoID, Nome, Categoria, Preco)
- ENCOMENDA(EncomendaID, *ClienteID*, Data, Estado)
- LINHA_ENCOMENDA(EncomendaID, ProdutoID, Quantidade) — com FK para ENCOMENDA e PRODUTO

1. Insira o cliente 20, Marta Gomes, de Braga, registada em 2026-03-01. *(1 val)*
2. Aumente 5 % o preço dos produtos da categoria *Acessórios* cuja quantidade total encomendada seja pelo menos 5. *(1,5 val)*
3. Apresente o nome do cliente, o número da encomenda, o nome do produto e a quantidade, para clientes de *Porto*, ordenado por número da encomenda e nome do produto. *(1,5 val)*

4. Apresente os produtos que nunca foram encomendados. *(1,5 val)*
5. Para cada cliente, apresente o nome, o número de encomendas e o valor total gasto (quantidade × preço), apenas para clientes com valor total superior a 100, por valor decrescente. *(2 val)*

Grupo III (5 val)

Uma cadeia de cinemas pretende analisar a venda de bilhetes por período temporal, cinema, filme e tipo de bilhete (normal, estudante, sénior).

1. Classifique como verdadeira ou falsa e justifique brevemente: *(1,5 val)*
   a) Um sistema OLAP é adequado para registar a venda de um bilhete no balcão.
   b) Na fase de transformação do ETL limpam-se e uniformizam-se os dados antes de os carregar no Data Warehouse.
   c) A percentagem de ocupação das salas pode ser somada entre cinemas, como qualquer medida aditiva.
   d) No esquema em estrela as dimensões ligam diretamente à tabela de factos.
2. Proponha um modelo dimensional: granularidade, principais medidas, pelo menos quatro dimensões com atributos. *(2 val)*
3. Identifique a operação OLAP (roll-up, drill-down, slice, dice ou pivot): *(1,5 val)*
   a) passar de vendas por trimestre para vendas por mês b) consultar só o filme "Duna" c) analisar os cinemas de Lisboa e Porto apenas para filmes de animação d) trocar as linhas (meses) e as colunas (tipos de bilhete) e) passar de vendas por cinema para vendas por cidade.

Soluções do exame de treino

Não leias antes de fazeres o exame

Grupo I — solução

1. Modelo ER

Entidades	Atributos e identificadores
ALUNO	NumAluno, Nome, DataNascimento, **{Telefone}** (multivalor)
PROFESSOR	ProfID, Nome, Email
INSTRUMENTO	CodInstrumento, Nome
AULA	NumAula, DataHora, Sala

Relacionamento	Tipo	(min,max)	Atributos / justificação
PROFESSOR – *leciona* – INSTRUMENTO	N:M	PROFESSOR (1,N) · INSTRUMENTO (0,N)	"pelo menos um instrumento"; instrumento pode não ter professor
PROFESSOR – *dá* – AULA	1:N	PROFESSOR (0,N) · AULA (1,1)	cada aula tem um único professor
AULA – *incide em* – INSTRUMENTO	N:1	AULA (1,1) · INSTRUMENTO (0,N)	cada aula tem um único instrumento
ALUNO – *frequenta* – AULA	N:M	ALUNO (0,N) · AULA (1,N)	**Classificacao** (opcional): atributo do relacionamento — depende do par aluno–aula

Regra de negócio não capturável só com este modelo: "o professor da aula leciona o instrumento da aula" — seria uma restrição adicional.

2. Esquema relacional

- ALUNO(NumAluno, Nome, DataNascimento)
- ALUNO_TELEFONE(NumAluno, Telefone) — NumAluno → ALUNO *(multivalor → tabela própria)*
- PROFESSOR(ProfID, Nome, Email)
- INSTRUMENTO(CodInstrumento, Nome)
- PROFESSOR_INSTRUMENTO(ProfID, CodInstrumento) — ProfID → PROFESSOR; CodInstrumento → INSTRUMENTO *(N:M)*
- AULA(NumAula, DataHora, Sala, *ProfID*, *CodInstrumento*) — ProfID → PROFESSOR; CodInstrumento → INSTRUMENTO *(duas 1:N; ambas NOT NULL)*
- FREQUENTA(NumAluno, NumAula, Classificacao) — NumAluno → ALUNO; NumAula → AULA *(N:M; Classificacao pode ser NULL)*

3. Integridade

Operação	Resposta
a) AULA com ProfID = 3	**Integridade referencial**: não existe professor 3.
b) PROFESSOR com ProfID = NULL	**Integridade de entidade**: a PK não pode ser NULL.
c) apagar o professor 1	**Integridade referencial**: há uma aula que o referencia (rejeitado, ou exige CASCADE/SET NULL — mas ProfID de AULA é obrigatório, logo SET NULL não serviria).
d) professor (3, Eva, ana@m.pt)	Viola a **unicidade da chave candidata Email** (repete o e-mail da Ana). Não é entidade (ProfID é novo e não nulo) nem referencial; é uma restrição de unicidade (UNIQUE).

Grupo II — solução

1. INSERT

```sql
INSERT INTO CLIENTE (ClienteID, Nome, Cidade, DataRegisto)
VALUES (20, 'Marta Gomes', 'Braga', '2026-03-01');
```

Estado da tabela depois de executar:

ClienteID	Nome	Cidade	DataRegisto
20	Marta Gomes	Braga	2026-03-01

2. UPDATE com subconsulta agregada

A condição "quantidade total encomendada ≥ 5" é sobre **grupos** de linhas de LINHA_ENCOMENDA → subconsulta com `GROUP BY … HAVING`. Só a "Capa de telemóvel" (3+2 = 5 unidades) cumpre; o "Carregador USB" só tem 3.

```sql
UPDATE PRODUTO
SET Preco = Preco * 1.05
WHERE Categoria = 'Acessórios'
  AND ProdutoID IN (
        SELECT ProdutoID
        FROM LINHA_ENCOMENDA
        GROUP BY ProdutoID
        HAVING SUM(Quantidade) >= 5
  );
```

Estado da tabela depois de executar:

ProdutoID	Nome	Categoria	Preco
1	Capa de telemóvel	Acessórios	13.13
2	Carregador USB	Acessórios	20
3	Auscultadores	Áudio	60
4	Coluna Bluetooth	Áudio	90
5	Cabo HDMI	Acessórios	8
6	Portátil	Computadores	800

3. Junção de 4 tabelas

```sql
SELECT C.Nome, E.EncomendaID, P.Nome AS Produto, LE.Quantidade
FROM CLIENTE AS C
JOIN ENCOMENDA AS E ON E.ClienteID = C.ClienteID
JOIN LINHA_ENCOMENDA AS LE ON LE.EncomendaID = E.EncomendaID
JOIN PRODUTO AS P ON P.ProdutoID = LE.ProdutoID
WHERE C.Cidade = 'Porto'
ORDER BY E.EncomendaID, P.Nome;
```

Resultado nos dados de exemplo:

Nome	EncomendaID	Produto	Quantidade
Inês Castro	1	Auscultadores	1
Inês Castro	1	Capa de telemóvel	3
Inês Castro	2	Capa de telemóvel	2
Inês Castro	2	Carregador USB	2
Lara Moura	4	Portátil	1

4. Anti-junção

```sql
SELECT P.ProdutoID, P.Nome
FROM PRODUTO AS P
WHERE NOT EXISTS (
        SELECT 1
        FROM LINHA_ENCOMENDA AS LE
        WHERE LE.ProdutoID = P.ProdutoID
);
```

Resultado nos dados de exemplo:

ProdutoID	Nome
5	Cabo HDMI

5. Junção + agregação com expressão

Para "número de encomendas" usa `COUNT(DISTINCT E.EncomendaID)`: como há várias linhas por encomenda, `COUNT(*)` contaria linhas de produto e não encomendas.

```sql
SELECT C.Nome,
       COUNT(DISTINCT E.EncomendaID)   AS NumEncomendas,
       SUM(LE.Quantidade * P.Preco)    AS ValorTotal
FROM CLIENTE AS C
JOIN ENCOMENDA       AS E  ON E.ClienteID   = C.ClienteID
JOIN LINHA_ENCOMENDA AS LE ON LE.EncomendaID = E.EncomendaID
JOIN PRODUTO         AS P  ON P.ProdutoID    = LE.ProdutoID
GROUP BY C.ClienteID, C.Nome
HAVING SUM(LE.Quantidade * P.Preco) > 100
ORDER BY ValorTotal DESC;
```

Resultado nos dados de exemplo:

Nome	NumEncomendas	ValorTotal
Lara Moura	1	800
Inês Castro	2	162.5
João Pires	1	110

Erros típicos. Q2: tentar filtrar a quantidade com `WHERE SUM(…)`. Q5: `COUNT(*)` em vez de `COUNT(DISTINCT …)`; esquecer que o valor vem de **duas tabelas** (Quantidade em LINHA_ENCOMENDA, Preco em PRODUTO); pôr `HAVING` antes do `GROUP BY`.

Bónus de treino — produtos com preço acima da média:

```sql
SELECT Nome, Preco
FROM PRODUTO
WHERE Preco > (SELECT AVG(Preco) FROM PRODUTO);
```

Resultado nos dados de exemplo:

Nome	Preco
Portátil	800

Grupo III — solução

1. Verdadeiro ou falso

Afirmação	V/F	Justificação
a) OLAP para registar a venda no balcão	**F**	Registar transações curtas é papel do **OLTP**; o OLAP é para análise multidimensional.
b) Transformação do ETL limpa e uniformiza	**V**	E-T-L: extrair, transformar (validar, limpar, uniformizar) e carregar.
c) % de ocupação somável entre cinemas	**F**	Medida **não aditiva**: recalcula-se (lugares ocupados ÷ lugares disponíveis) no nível pretendido.
d) Estrela: dimensões ligadas diretamente aos factos	**V**	Uma tabela de factos central ligada diretamente a cada dimensão.

2. Modelo dimensional

- **Grão:** um bilhete vendido (uma linha por bilhete).
- **Medidas: NumBilhetes** (=1), **Receita** (preço pago), **Desconto** — todas aditivas ao nível do bilhete. A % de ocupação seria **não aditiva**.
- **FACT_BILHETE**: TempoKey, CinemaKey, FilmeKey, SessaoKey, TipoBilheteKey (FK) + medidas.
- **DIM_TEMPO**: data, dia, mês, trimestre, ano · **DIM_CINEMA**: nome, cidade, distrito · **DIM_FILME**: título, género, duração, classificação etária · **DIM_SESSAO**: horário, tipo de sala (2D/3D) · **DIM_TIPO_BILHETE**: normal / estudante / sénior (preço base).
- **Hierarquias:** dia → mês → trimestre → ano; cinema → cidade → distrito.

3. Operações OLAP

Situação	Operação
a) trimestre → mês	**Drill-down** (mais detalhe)
b) só o filme "Duna"	**Slice** (um valor numa dimensão)
c) Lisboa e Porto, só animação	**Dice** (vários valores e duas dimensões)
d) trocar meses (linhas) e tipos de bilhete (colunas)	**Pivot**
e) cinema → cidade	**Roll-up** (menos detalhe)

5 · Reta final

Folha A4, erros típicos, glossário e dados de exemplo

5.1 O que pôr na folha A4 (frente e verso, manuscrita)

A folha tem de ser **escrita à mão e feita por ti** — não podes imprimir nem fotocopiar. Usa-a para o que é difícil de memorizar de forma exata e para esquemas que poupam tempo. Sugestão de conteúdo, por ordem de utilidade:

Frente — Módulo 1 + relacional

- Notação ER: entidade, atributo, identificador (sublinhado), multivalor (oval dupla), composto, derivado (tracejado), relacionamento, (min,max).
- Como ler (min,max): "uma instância de A tem quantas de B?" → escreve junto de A.
- Tabela ER → relacional: 1:1, 1:N (FK no lado N), N:M (nova tabela, PK composta), recursivo, multivalor (tabela), composto (colunas), derivado (não se guarda), ternário.
- Definições: chave candidata (única + mínima), PK, FK; integridade de domínio / entidade / referencial com 1 exemplo cada.
- Método em 8 passos (secção 1.4).
- Exemplo mínimo EMPRESTIMO_LIVRO(EmprestimoID, ISBN, Estado).

Verso — SQL + Módulo 3

- Molde do SELECT com a ordem das cláusulas; ordem lógica (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT).
- INSERT / UPDATE (SET…WHERE) / DELETE; **WHERE sempre**.
- JOIN … ON; LEFT JOIN … IS NULL; NOT EXISTS (modelo); IN (subconsulta); NOT IN + NULL.
- WHERE vs HAVING; COUNT(*) vs COUNT(col); regra do GROUP BY.
- BETWEEN (inclusivo), IN, LIKE (% e _), IS NULL.
- OLTP vs OLAP (tabela), DW, ETL (E-T-L), 3 V, ROLAP/MOLAP/HOLAP.
- Grão → dimensões → medidas; aditiva / semi / não aditiva; star / snowflake / galaxy.
- Roll-up / drill-down / slice / dice / pivot (com 1 exemplo cada).

> **Dica.** Põe na folha **coisas que costumas confundir** (WHERE vs HAVING; slice vs dice; FK no lado N) em vez de copiar definições que já sabes. E treina o exame de treino **com a folha que vais levar**, para ver o que falta.

5.2 Erros típicos, por grupo

Grupo	Os erros que mais custam pontos
I · ER / relacional	FK dentro do ER · N:M sem tabela de associação · atributo do relacionamento posto numa das entidades · FK no lado errado do 1:N · PK sem composição na tabela N:M · falta de (min,max) · esquecer de dizer que criaste um identificador · não assinalar PK/FK · confundir "candidata" (única+mínima) com "super-chave".
II · SQL	Sem aspas em texto · UPDATE/DELETE sem WHERE · JOIN sem ON · coluna ambígua sem alias · WHERE com agregação (usar HAVING) · GROUP BY incompleto · "nenhum/nunca" resolvido com INNER JOIN · NOT IN com NULL · **>** vs **>=** · ORDER BY sem DESC quando pedem decrescente.
III · BI / OLAP	Trocar OLTP e OLAP · esquecer a granularidade (ou dizê-la como dimensão) · menos de 4 dimensões / sem atributos · somar médias e percentagens · confundir slice (um valor) com dice (combinação) e roll-up com drill-down.

5.3 Plano de estudo sugerido

Fase	O quê
1. Rever	Ler os resumos das Partes 1–3 e fazer a folha A4.
2. Refazer o modelo	Resolver à mão o exame modelo (sem ler a resolução); só depois comparar.
3. Variações	Casos A–C e exercícios I–VI (Módulo 1); as variações de SQL (escreve antes de ver); V1–V6 (Módulo 3).
4. Simulacro	Exame de treino em 90 min com a folha A4. Anota onde perdes tempo.
5. Véspera	Reler a checklist (5.2) e só a tua folha. Dormir.

5.4 Glossário PT–EN

Os slides do Módulo 1 e 2 estão em inglês; o exame modelo está em português. Convém reconhecer os dois.

Português	English	Português	English
Entidade	Entity	Chave primária / estrangeira	Primary / foreign key
Atributo (composto, multivalor, derivado)	Attribute (composite, multivalued, derived)	Chave candidata	Candidate key
Relacionamento	Relationship	Integridade de entidade / referencial / de domínio	Entity / referential / domain integrity
Identificador	Identifier	Participação total / parcial	Total / partial participation
Cardinalidade	Cardinality	Relacionamento recursivo	Recursive relationship
Tuplo / linha	Tuple / row	Relação / tabela	Relation / table
Tabela de factos	Fact table	Tabela de dimensão	Dimension table
Medida	Measure	Granularidade	Granularity (grain)
Esquema em estrela / floco de neve / galáxia	Star / snowflake / galaxy schema	Armazém de dados	Data Warehouse
Extração, transformação, carregamento	Extract, Transform, Load	Cubo	Cube
Detalhar / resumir	Drill-down / roll-up	Fatiar / cortar em dados / rodar	Slice / dice / pivot

Anexo A — Fontes e limitações

- **Fontes:** "01 Aula" (Módulo 1, 57 slides), "02 Aula" (Módulo 2, 41 slides), "03 Aula" (Módulo 3, 38 slides), Exame modelo, Resolução do exame modelo, email da professora.
- **Verificação:** todo o SQL e todos os resultados foram executados em SQLite 3.45 sobre dados inventados (Anexo B). Soluções de ER/relacional/dimensional são **soluções possíveis**, como a própria resolução oficial diz.
- **Fora dos slides:** o que está marcado extra / extensão (IS NULL, MIN/MAX, LEFT JOIN em detalhe, ternário, ponte para N:M no dimensional) foi acrescentado porque as soluções oficiais ou variações naturais o exigem; confirma com a professora se entram na matéria.
- **Fora do âmbito destes ficheiros:** normalização, generalização/especialização, índices em detalhe, ALTER TABLE.

- As “variações” e o exame de treino são **invenção minha** para praticar — o email diz que o exame real pode ter outros contextos e distribuição.

Anexo B — Dados de exemplo (manutenção de bicicletas)

Cola isto numa ferramenta SQLite (DB Browser for SQLite, `sqlite3` na linha de comandos ou um editor online) para testar as consultas da Parte 2. Os restantes dados (biblioteca, estrela de entregas, loja online) estão no ficheiro `dados_de_exemplo.sql` que acompanha este guia, e a loja online também no Anexo C.

```
CREATE TABLE TECNICO (
  TecnicoID TEXT PRIMARY KEY,
  Nome TEXT NOT NULL,
  Certificacao TEXT,
  AnosExperiencia INTEGER,
  Especialidade TEXT
);
CREATE TABLE MANUTENCAO (
  ManutencaoID INTEGER PRIMARY KEY,
  Descricao TEXT,
  TecnicoID TEXT,
  Ano INTEGER,
  Tipo TEXT,
  Custo REAL,
  FOREIGN KEY (TecnicoID) REFERENCES TECNICO(TecnicoID)
);
CREATE TABLE CATEGORIA (
  CategoriaID INTEGER PRIMARY KEY,
  NomeCategoria TEXT NOT NULL
);
CREATE TABLE MANUTENCAO_CATEGORIA (
  ManutencaoID INTEGER,
  CategoriaID INTEGER,
  PRIMARY KEY (ManutencaoID, CategoriaID),
  FOREIGN KEY (ManutencaoID) REFERENCES MANUTENCAO(ManutencaoID),
  FOREIGN KEY (CategoriaID) REFERENCES CATEGORIA(CategoriaID)
);
INSERT INTO TECNICO VALUES
 ('T01','Ana Sousa','Shimano',8,'Transmissão'),
 ('T02','Bruno Reis','SRAM',3,'Transmissão'),
 ('T03','Carla Dias','Shimano',12,'Travões'),
 ('T04','Diogo Faria',NULL,1,'Rodas'),
 ('T05','Eva Matos','SRAM',5,'Travões'),
 ('T06','Filipe Cruz','Shimano',2,'Suspensão');
INSERT INTO CATEGORIA VALUES (1,'Mecânica'),(2,'Segurança'),(3,'Estética'),(4,'Elétrica');
INSERT INTO MANUTENCAO VALUES
 (1,'Troca de corrente','T01',2025,'Corretiva',120.00),
 (2,'Afinação de mudanças','T02',2025,'Preventiva',45.00),
 (3,'Substituição de pastilhas','T03',2026,'Corretiva',150.00),
 (4,'Revisão de travões','T05',2026,'Preventiva',80.00),
 (5,'Troca de cassete','T01',2026,'Corretiva',210.00),
 (6,'Centragem de roda','T04',2025,'Corretiva',30.00),
 (7,'Troca de desviador','T02',2026,'Corretiva',95.00),
 (8,'Revisão geral','T03',2026,'Preventiva',100.00);
INSERT INTO MANUTENCAO_CATEGORIA VALUES (1,1),(2,1),(3,2),(3,1),(4,2),(5,1),(6,1),(7,1),(8,1),
(8,2);
```

Anexo C — Dados de exemplo (loja online, exame de treino)

```
CREATE TABLE CLIENTE (ClienteID INTEGER PRIMARY KEY, Nome TEXT NOT NULL, Cidade TEXT, DataRegisto
TEXT);
CREATE TABLE PRODUTO (ProdutoID INTEGER PRIMARY KEY, Nome TEXT NOT NULL, Categoria TEXT, Preco
REAL);
CREATE TABLE ENCOMENDA (EncomendaID INTEGER PRIMARY KEY, ClienteID INTEGER NOT NULL, Data TEXT,
Estado TEXT,
  FOREIGN KEY (ClienteID) REFERENCES CLIENTE(ClienteID));
CREATE TABLE LINHA_ENCOMENDA (EncomendaID INTEGER, ProdutoID INTEGER, Quantidade INTEGER NOT NULL,
  PRIMARY KEY (EncomendaID, ProdutoID),
  FOREIGN KEY (EncomendaID) REFERENCES ENCOMENDA(EncomendaID),
  FOREIGN KEY (ProdutoID) REFERENCES PRODUTO(ProdutoID));
INSERT INTO CLIENTE VALUES (1,'Inês Castro','Porto','2025-06-01'),(2,'João Pires','Lisboa','2025-
09-12'),(3,'Lara Moura','Porto','2026-01-20'),(4,'Nuno Vaz','Faro','2026-02-02');
INSERT INTO PRODUTO VALUES (1,'Capa de telemóvel','Acessórios',12.50),(2,'Carregador
USB','Acessórios',20.00),(3,'Auscultadores','Áudio',60.00),(4,'Coluna Bluetooth','Áudio',90.00),
(5,'Cabo HDMI','Acessórios',8.00),(6,'Portátil','Computadores',800.00);
INSERT INTO ENCOMENDA VALUES (1,1,'2026-02-10','Entregue'),(2,1,'2026-03-01','Enviada'),
(3,2,'2026-03-05','Entregue'),(4,3,'2026-03-08','Pendente');
INSERT INTO LINHA_ENCOMENDA VALUES (1,1,3),(1,3,1),(2,2,2),(2,1,2),(3,4,1),(3,2,1),(4,6,1);
```